# Journal de bord

Mondes vivants · Sujet 04 ÉPHÉMÈRE · Simulation multi-agents du lancement d'une crème solaire fictive

> **À quoi sert ce document.** Après chaque étape du développement, on y explique ce qui a été fait, pourquoi, et ce qu'il faut retenir pour pouvoir le présenter au jury. Il se lit dans l'ordre : chaque étape s'appuie sur les précédentes. Les règles permanentes du projet sont dans `CLAUDE.md`, les décisions de conception dans `docs/contexte-projet.md`.

## Le plan de travail

Le projet repart de zéro le 16 octobre 2026. Chaque étape est validée par le binôme avant de passer à la suivante, puis fait l'objet d'un commit. Les codes entre parenthèses renvoient aux exigences de l'énoncé.

| # | Étape | Contenu | État |
|---|---|---|---|
| **Phase 0 — Faire place nette** | | | |
| 0 | Nettoyage du dépôt | Retrait du prototype non conforme, mise à jour de CLAUDE.md, README et journal | terminée |
| **Phase 1 — Fondations** | | | |
| 1 | Squelette du dépôt | Arborescence, `.gitignore`, `.env.example`, Python 3.12, ruff, fichier de configuration | terminée |
| 2 | Infrastructure Docker | Kafka (KRaft) et Redis en versions figées, création des 7 topics `sim.*` | terminée |
| 3 | Mesure de la machine | Réglages Ollama, benchmark par modèle en séquentiel puis en parallèle (E4) | terminée |
| **Phase 2 — Le cœur, sans contenu du monde** | | | |
| 4 | Contrat d'événements | Enveloppe Pydantic versionnée, deux horloges, `caused_by`, file `sim.dlq` (F4, C3, F6) | terminée |
| 5 | Horloge et orchestrateur | Mode synchrone, délai de livraison, `run_id` et graine (C2, C3) | terminée |
| 6 | Couche LLM | Sortie JSON imposée, routeur par palier, relance puis repli, incidents, coûts (A4, A5, E4) | terminée |
| 7 | Tranche verticale | 3 agents LLM et 1 agent à règles sur 5 pas, lus par un client console | terminée |
| 8 | Historique et rejeu | Écriture Parquet, relecture d'un run sans appel LLM (E3, F5) | terminée |
| **Phase 3 — Visualisation** | | | |
| 9 | Pont FastAPI | WebSocket en direct ou en rejeu, inspection (D2), injection (D3) | terminée |
| 10 | Scène Godot 3D | Bâtiments distincts par zone, plage avec sable et mer, personnages animés, panneau d'indicateurs (D1) | terminée |
| **Phase 4 — Le monde et ses agents** | | | |
| 11 | Le monde | Nom de marque et test de contamination, population complète, seuils d'adoption | terminée |
| 12 | Corpus et RAG | Documents fictifs, ChromaDB, filtres par date et par rôle, mémoires, évaluation (F1, F2, A1, C6) | terminée |
| 13 | Agents à règles et baseline | Météo, vendeurs, concurrent, fidèles ; baseline C0 avec la même saisonnalité (A6, E1) | terminée |
| 14 | Indicateurs | Sens partagé, adoption, traçabilité, run témoin, taux de sortie de rôle (B1-B3, F6) | terminée |
| 15 | Mode asynchrone | Deuxième mode d'exécution, violations temporelles comptées (C4, C6) | terminée |
| **Phase 5 — Preuves et soutenance** | | | |
| 16 | Expériences | Configurations C0 à C3, au moins 5 runs chacune, Bass, synchrone/asynchrone, coûts (E1, E2, E4, E5, C5) | à venir |
| 17 | Charge et pannes | Agent muet, pic d'événements, latence jusqu'aux indicateurs (O4, C7) | à venir |

| | | | |
|---|---|---|---|
| 18 | Soutenance | Configuration de démo réduite, deux tailles de population (C8), README, run enregistré, rapport | à venir |

**Pourquoi cet ordre.** On construit d'abord la tuyauterie (événements, horloge, appels LLM) avec très peu d'agents, pour vérifier que tout circule. Le rejeu arrive tôt (étape 8) parce qu'il permet ensuite de développer la scène 3D sur des runs enregistrés, sans occuper la carte graphique avec les modèles. Le contenu du monde (marque, corpus, population complète) vient une fois la chaîne fiable.

# Étape 0 — Nettoyage du dépôt

16 octobre 2026 · Exigences concernées : aucune directement ; prépare toutes les suivantes.

### Objectif

Repartir d'une base propre : ne garder dans le dépôt que ce qui est conforme aux règles du projet ( `CLAUDE.md` ), aux décisions de conception ( `docs/contexte-projet.md` ) et à l'énoncé.

### Pourquoi on a repris de zéro

Un premier prototype avait été écrit avant que la conception soit arrêtée. En le comparant aux règles du projet, plusieurs écarts sont apparus, dont certains touchent des exigences notées :

| Écart dans le prototype | Règle ou exigence concernée |
|---|---|
| Nom de marque, ingrédient et indice de protection écrits en dur dans le code et les prompts | Le nom doit rester un paramètre de configuration, choisi après un test de contamination |
| Topics `simulation.*` au lieu des 7 topics `sim.*` prévus | C1, architecture |
| Événements sans `run_id` , sans `caused_by` , et date de traitement en temps réel au lieu du temps simulé | C3 (deux horloges), F6 (traçabilité), E3 (rejeu) |
| Pas de Redis ; images Docker en version `latest` | F3 ; livrable reproductible sur une autre machine |
| Pas de `.gitignore` , fichiers compilés Python versionnés, arborescence différente | Conventions du dépôt |
| Visualisation 2D à base d'icônes | Remplacée par une scène 3D (décision du binôme) |

Corriger ces écarts un par un aurait coûté plus cher que de reconstruire proprement, étape par étape.

### Ce qui a été fait

- **Retrait du prototype du dépôt** : `src/` , `godot_client/` , `docker-compose.yml` , `requirements.txt` . Ces fichiers avaient été commités, ils restent donc consultables dans l'historique git (commit `607297b` ).
- **Archivage hors du dépôt** des fichiers jamais commités, qu'une suppression aurait perdus définitivement : `src/agents/` , `src/rag/` , `src/simulation_engine.py` , `data/knowledge/` , les caches Godot et l'environnement virtuel. Ils sont dans `C:\Users\Siradiou\archive_prototype_S9_2026-10-16\` .
- **Mise à jour de `CLAUDE.md`** : chemin correct de l'énoncé, Python 3.12, Godot en 3D low-poly, et une section « Méthode de travail » qui fixe la règle d'avancement étape par étape.
- **Journal des décisions** complété dans `docs/contexte-projet.md` .
- **Création du `README.md`** : description du projet pour GitHub et tableau d'avancement, mis à jour à chaque commit.
- **Installation de Python 3.12.10** avec le gestionnaire d'installation officiel ( `py install 3.12` ), pour l'utilisateur seulement, sans droits administrateur. Python 3.14 reste installé et reste la version par défaut ; on appelle 3.12 explicitement avec `py -V:3.12` .
- **Création de ce journal de bord** : source HTML dans `docs/journal/journal-de-bord.html` , PDF généré à la racine par `docs/journal/build.ps1` (impression via Microsoft Edge, déjà présent sous Windows, donc aucune dépendance à installer).

### Décisions prises à cette étape

| Décision | Raison |
|---|---|
| Python **3.12** | Version stable bien prise en charge par les bibliothèques prévues (ChromaDB notamment), qui suivent souvent les nouvelles versions de Python avec retard. Python 3.12.10 a été installé à côté de 3.14 (voir ci-dessous). |
| Scène Godot en **3D low-poly**, rendu léger | Plus lisible et plus parlante pour le jury, tout en laissant de la mémoire graphique aux modèles LLM (budget d'environ 1 à 1,5 Go pour Windows et Godot). |
| Modèles 3D gratuits choisis sur maquettes | Plusieurs essais seront présentés au binôme, qui choisira (étape 10). |
| Avancement étape par étape, commit après validation | Chaque étape est comprise et expliquable par les deux membres avant d'aller plus loin. |

### État du dépôt après l'étape

```
CLAUDE.md                  règles permanentes du projet
README.md                  présentation et état d'avancement
journal-de-bord.pdf        ce document
docs/
  contexte-projet.md       décisions de conception
  Architecture_…pdf        schémas d'architecture
  Mondes-Vivants_…html     énoncé
  journal/
    journal-de-bord.html   source de ce document
    build.ps1              génération du PDF
```

### À retenir pour l'oral

- Le projet a été reconstruit sur une conception écrite et validée avant de coder, pour que chaque composant réponde à une exigence identifiée de l'énoncé.
- Interpréteur et environnement virtuel sont deux choses distinctes : Python 3.12 est installé une fois sur la machine, puis l'environnement virtuel `.venv` du projet (étape 1) est créé à partir de lui et contient toutes les bibliothèques. Ainsi les dépendances du projet n'interfèrent pas avec le reste de la machine, et la procédure se reproduit sur la machine de la collaboratrice.
- Rien n'a été perdu : l'ancien code est dans l'historique git ou archivé.

### Prochaine étape

Étape 1 — Squelette du dépôt : environnement virtuel `.venv` en Python 3.12, arborescence, `.gitignore`, `.env.example`, outillage ruff, et fichier de configuration qui portera `BRAND_NAME`, le nombre d'agents, le mode d'exécution et la graine.

# Étape 1 — Squelette du dépôt

16 octobre 2026 · Exigences concernées : C2 (nombre d'agents fixé au lancement), livrable « reproductible sur une autre machine, versions figées ».

### Objectif

Poser le cadre dans lequel tout le code viendra se ranger : un environnement Python isolé et reproductible, une arborescence qui suit l'architecture, des outils de qualité, et un fichier de configuration qui centralise tous les paramètres d'un run.

### Ce qui a été fait

- **Environnement virtuel** `.venv` créé à partir de Python 3.12.10. Toutes les bibliothèques du projet y sont installées, rien sur le reste de la machine.
- **Arborescence** : un dossier par composant de l'architecture ( `sim`, `agents`, `llm`, `bus`, `rag`, `indicators`, `bridge`, `replay` ), plus `corpus`, `godot`, `configs` et `tests`. Chaque paquet Python porte une phrase qui dit à quoi il sert.
- `pyproject.toml` : fiche d'identité du projet. Il déclare les dépendances directes avec leur version exacte, configure ruff (style et erreurs courantes) et pytest.
- `requirements.lock` : liste complète et figée de toutes les bibliothèques installées, y compris les dépendances indirectes.
- `.gitignore` : exclut l'environnement virtuel, les caches, le fichier de secrets `.env`, les sorties de runs et le cache de Godot.
- `.env.example` : modèle du fichier de secrets, sans valeur (adresses d'Ollama, Kafka, Redis ; clés d'API facultatives).
- **Configuration** : `configs/default.yaml` et son chargeur `sim/config.py`, plus la liste des 11 rôles dans `agents/roles.py`.
- **11 tests automatisés** dans `tests/test_config.py`.

### Décisions prises à cette étape

| Décision | Raison |
|---|---|
| pip + fichier verrou | Outils standards de Python, rien à apprendre en plus. Le verrou fige aussi les dépendances indirectes : la collaboratrice obtient exactement les mêmes versions. |
| Configuration en YAML, validée par Pydantic | Lisible et commentable, prévu dans le document d'architecture. La validation transforme une faute de frappe en erreur claire au lancement plutôt qu'en comportement faux au milieu d'un run. |
| Une dépendance n'arrive qu'avec l'étape qui l'utilise | Chaque bibliothèque du projet est justifiée et expliquable. Ainsi `python-dotenv` (lecture de `.env` ) attendra l'étape 2. |
| Nom de marque provisoire `MARQUE_PROVISOIRE` | Le vrai nom sera choisi à l'étape 11 après le test de contamination. Comme il n'existe que dans la configuration, le changer ne touchera aucun code. |

### Comment fonctionne la configuration

Un run est entièrement décrit par un fichier YAML en quatre sections :

| Section | Contenu | Valeur par défaut |
|---|---|---|
| `run` | Graine du générateur aléatoire | 42 |
| `simulation` | Durée simulée en jours, mode d'exécution | 90 jours, synchrone |
| `monde` | Nom de marque, ingrédient phare, indice de protection (fictifs) | valeurs provisoires |
| `population` | Nombre d'agents pour chacun des 11 rôles | 120 agents, dont 84 LLM |

Au chargement, `load_config` vérifie chaque valeur. Sont refusés : une durée nulle, une graine négative, un mode autre que `synchrone` ou `asynchrone`, un nom de marque vide, un rôle inconnu, un effectif négatif, une population vide, et toute clé mal orthographiée (par exemple `duree_jour` au lieu de `duree_jours`). L'erreur indique le fichier et le champ fautif.

> **Lien avec l'exigence C2.** Pour changer le nombre d'agents, on modifie la section `population` du fichier YAML, ou on crée un autre fichier dans `configs/`. Le code ne change pas. Un test le vérifie avec une population de 4 agents.

### Les tests

| Test | Ce qu'il garantit |
|---|---|
| `test_default_config_matches_design` | La configuration par défaut correspond à la conception : 120 agents, 90 jours, mode synchrone, les 11 rôles présents. |
| `test_agent_count_comes_from_config` | Le nombre d'agents vient bien du fichier (C2). |
| `test_invalid_values_are_rejected` (7 cas) | Chaque valeur invalide listée plus haut est refusée. |
| `test_empty_population_is_rejected` | Un run sans agent est refusé. |
| `test_missing_file_gives_clear_error` | Un fichier absent donne une erreur lisible. |

Résultat : 11 tests réussis, aucune remarque de ruff. La procédure d'installation du README a été rejouée dans un environnement neuf pour vérifier qu'elle fonctionne telle qu'écrite.

### État du dépôt après l'étape

```
pyproject.toml        identité du projet, dépendances directes, ruff, pytest
requirements.lock     toutes les versions figées
.gitignore  .env.example
configs/default.yaml  configuration principale (120 agents, 90 jours)
sim/config.py         chargement et validation de la configuration
agents/roles.py       les 11 rôles
tests/test_config.py  11 tests
sim/ agents/ llm/ bus/ rag/ indicators/ bridge/ replay/   paquets Python
corpus/ godot/        à remplir aux étapes 10 et 12
```

### À retenir pour l'oral

- Un run est entièrement décrit par un fichier de configuration et une graine : c'est la base de la reproductibilité (E2, E3) et de l'exigence C2.
- La configuration est validée avant le lancement : une erreur de saisie ne peut pas fausser un run silencieusement.
- Les versions sont figées à deux niveaux (`pyproject.toml` pour les dépendances directes, `requirements.lock` pour toutes), ce qui répond à l'exigence de livrable reproductible sur une autre machine.

### Prochaine étape

Étape 2 — Infrastructure Docker : Kafka en mode KRaft et Redis, versions figées, création des 7 topics `sim.*`, et vérification que Python peut y écrire et y lire.

# Étape 2 — Infrastructure Docker : Kafka et Redis

16 octobre 2026 · Exigences concernées : C1 (événements transportés par Kafka), F3 (où vit chaque donnée), livrable reproductible.

### Objectif

Mettre en place les deux services qui tournent à côté de la simulation : **Kafka**, le bus par lequel passent tous les événements, et **Redis**, qui garde l'état courant du monde. Vérifier que le code Python sait y écrire et y relire.

### Rappel : à quoi servent ces deux services

| Service | Rôle dans le projet | Pourquoi lui |
|---|---|---|
| **Kafka** | Journal ordonné de tous les événements : décisions des agents, messages, météo, indicateurs, incidents. Les agents ne s'appellent jamais directement, ils publient et lisent des événements. | Imposé par l'énoncé (C1). Garde l'ordre, permet de relire un flux, découple la simulation de Godot et des indicateurs. |
| **Redis** | État courant : stocks, météo, qui est dans quelle zone. Lu et écrit très souvent par les agents et par le pont vers Godot. | Base en mémoire, réponse en moins d'une milliseconde. Les données sont éphémères (F3). |

### Ce qui a été fait

- `docker-compose.yml` : décrit les deux conteneurs, avec des versions exactes (`apache/kafka:4.3.1`, `redis:8.8.3-alpine`) et un contrôle de santé pour chacun. `docker compose up -d --wait` les lance et attend qu'ils soient prêts.
- `bus/topics.py` : la liste des 7 topics sous forme d'énumération Python, et la commande `python -m bus.topics` qui crée ceux qui manquent. Elle peut être relancée sans risque : un topic déjà présent n'est pas recréé.
- `sim/settings.py` : lit les adresses des services dans `.env`. Sans fichier `.env`, les valeurs par défaut conviennent pour un poste local.
- **Trois bibliothèques ajoutées** : `aiokafka` (client Kafka compatible avec `asyncio`, que l'orchestrateur utilisera), `redis` (client Redis, avec une version `asyncio`) et `python-dotenv` (lecture de `.env`).
- **5 nouveaux tests**, dont 3 tests d'intégration qui parlent aux vrais services.

### Les 7 topics

| Topic | Ce qui y passe | Exigences |
|---|---|---|
| `sim.world.events` | Événements du monde : météo, stocks, promotions, événements injectés | C1, D3 |
| `sim.agent.messages` | Échanges en langage naturel entre agents | A2 |
| `sim.agent.decisions` | Action et sens déclarés par chaque agent | B1, D2 |
| `sim.indicators` | Indicateurs calculés pendant le run | B2, F6 |
| `sim.control` | Commandes et injections depuis l'interface | D3 |
| `sim.incidents` | Sorties LLM invalides, replis, violations temporelles | A4, C6 |
| `sim.dlq` | Événements qui ne respectent pas leur schéma (« dead letter queue ») | F4 |

### Choix de configuration et leurs raisons

| Choix | Raison |
|---|---|
| Kafka en mode **KRaft**, un seul nœud | Depuis Kafka 4, ZooKeeper n'existe plus : Kafka gère lui-même ses métadonnées. Un seul nœud suffit sur un portable. |
| **Une partition** par topic | Kafka ne garantit l'ordre qu'à l'intérieur d'une partition. Avec une seule partition, tous les événements d'un topic sont relus exactement dans l'ordre d'écriture, ce qui est indispensable pour rejouer un run à l'identique (E3). Le débit d'une partition dépasse de loin nos quelques dizaines de milliers d'événements par run. |
| Rétention de **7 jours** | Kafka sert de transport, pas d'archive. L'historique durable sera écrit en Parquet à l'étape 8 (F3). |
| Création automatique des topics **désactivée** | Par défaut, Kafka crée un topic dès qu'on écrit dedans. Une faute de frappe dans un nom créerait alors un topic fantôme où les événements se perdraient sans erreur. Désactivée, l'erreur est immédiate. |
| Mémoire de Kafka limitée à **1 Go** | Laisse la RAM aux autres composants. Mesuré au repos : 360 Mo pour Kafka, 5 Mo pour Redis. |
| Redis **sans persistance** sur disque | L'état courant est reconstruit à chaque run ; le sauvegarder ne servirait à rien et ralentirait les écritures. |
| Ollama **hors de Docker** | Sous Windows, donner le GPU à un conteneur ajoute de la complexité sans bénéfice (décision du document d'architecture). |

### Les tests

| Test | Ce qu'il garantit |
|---|---|
| `test_all_topics_exist_with_expected_settings` | Les 7 topics existent, chacun avec une seule partition. |
| `test_kafka_roundtrip_preserves_order` | 20 messages écrits dans un topic temporaire sont relus à l'identique et dans le même ordre ; le topic est supprimé ensuite. |
| `test_redis_roundtrip` | Une écriture dans Redis est relue correctement. |
| `test_defaults_match_env_example` | Sans `.env`, les adresses par défaut sont utilisées. |
| `test_env_file_and_environment` | `.env` est bien lu, et une variable d'environnement le remplace si elle est définie. |

Résultat : 16 tests réussis, aucune remarque de ruff. On a aussi vérifié que, si les services sont arrêtés, les tests d'intégration sont ignorés avec le message « lancer `docker compose up -d` » au lieu d'échouer.

### État du dépôt après l'étape

```
docker-compose.yml       Kafka 4.3.1 (KRaft) + Redis 8.8.3, versions figées
bus/topics.py            les 7 topics et leur création
sim/settings.py          adresses des services, lues dans .env
tests/test_infra.py      3 tests d'intégration (Kafka, Redis)
tests/test_settings.py   2 tests de lecture de .env
```

### À retenir pour l'oral

- Kafka est le seul moyen de communication entre composants : c'est ce qui rend un run enregistrable, rejouable et observable sans modifier la simulation.
- Une partition par topic donne un ordre total des événements : c'est un choix délibéré au service du rejeu, pas une limite.
- Chaque stockage a un rôle et une durée de vie : Kafka transporte (7 jours), Redis garde le présent (le temps d'un run), Parquet gardera l'historique (étape 8).

**Prochaine étape**

Étape 3 — Mesure de la machine : vérifier Ollama et les modèles Qwen3, régler ses variables (modèles gardés en mémoire, requêtes parallèles), puis mesurer la latence de chaque modèle en séquentiel et en parallèle. Ces mesures remplaceront les durées supposées du document d'architecture et serviront de base au coût (E4).

# Étape 3 — Mesure de la machine

16 octobre 2026 · Exigences concernées : E4 (coût d'un run), E5 (sobriété), A5 (modèles de tailles différentes), et l'objectif O4 (identifier le facteur limitant).

### Objectif

Remplacer les durées *supposées* du document d'architecture par des durées *mesurées* sur la machine du projet, avant d'écrire le moindre agent. On sait ainsi combien de temps durera un run, combien de mémoire graphique prend chaque modèle, et ce que coûte le passage d'un palier à l'autre.

### Ce qui a été fait

- **Téléchargement des trois modèles** Qwen3 : `qwen3:1.7b` (1,4 Go), `qwen3:4b` (2,5 Go), `qwen3:8b` (5,2 Go). Le premier réseau plafonnait à 140 Ko/s ; le changement de réseau a permis de finir à environ 10 Mo/s.
- **Réglage d'Ollama** par six variables d'environnement utilisateur (tableau ci-dessous), prises en compte au redémarrage d'Ollama.
- **Script de benchmark** `llm/benchmark.py`, relançable sur n'importe quelle machine (`python -m llm.benchmark`), y compris celle de la collaboratrice.
- **Résultats enregistrés** dans `docs/mesures/` : un résumé lisible (Markdown) et toutes les mesures brutes, appel par appel (JSON).
- **Bibliothèques ajoutées** : `ollama` (client officiel, qui servira aussi à l'étape 6) et `httpx` (requêtes HTTP simples).

### Les réglages d'Ollama

| Variable | Valeur | Raison |
|---|---|---|
| `OLLAMA_MAX_LOADED_MODELS` | 3 | Petit, moyen et modèle d'embeddings doivent rester chargés ensemble. Le document d'architecture proposait 2, mais le modèle d'embeddings compte aussi comme un modèle. |
| `OLLAMA_NUM_PARALLEL` | 4 | Jusqu'à 4 requêtes traitées en même temps par un modèle. |
| `OLLAMA_KEEP_ALIVE` | 30m | Un modèle reste en mémoire 30 minutes après son dernier appel, pour éviter des rechargements. |
| `OLLAMA_FLASH_ATTENTION` | 1 | Calcul de l'attention plus rapide et plus économe ; nécessaire pour compresser le cache. |
| `OLLAMA_KV_CACHE_TYPE` | q8_0 | Le cache de contexte est stocké sur 8 bits au lieu de 16 : deux fois moins de VRAM. |
| `OLLAMA_CONTEXT_LENGTH` | 2048 | Un prompt d'agent fait environ 650 tokens : 2 048 suffit et réserve moins de VRAM par requête. |

### Comment on a mesuré

Chaque appel imite une **décision d'agent** : un prompt d'environ 650 tokens (profil, jour, zone, météo, trois messages reçus, quatre passages « retrouvés »), une réponse JSON imposée (`action`, `sens`, `message`, `justification`), la réflexion de Qwen3 désactivée. Les prompts varient d'un appel à l'autre, mais de façon déterministe, pour que le benchmark soit reproductible. Le nom de marque vient de la configuration, jamais du code.

Pour chaque modèle : un chargement à froid, 12 appels l'un après l'autre, puis 12 appels par paquets de 2 et de 4 en parallèle. Pendant chaque série, `nvidia-smi` relève quatre fois par seconde la puissance, la mémoire et la température du GPU. On mesure enfin la **bascule de palier** : charger le 8b alors que le 1.7b et le 4b sont en mémoire, puis revenir au 1.7b.

### Résultats

| Mesure | qwen3:1.7b | qwen3:4b | qwen3:8b |
|---|---|---|---|
| VRAM occupée (modèle + cache pour 4 requêtes) | 1,7 Go | 3,1 Go | 5,4 Go |
| Part du modèle sur le GPU | 100 % | 100 % | 100 % |
| Latence médiane d'une décision, en séquentiel | 0,76 s | 1,49 s | 2,14 s |
| Latence supposée dans le document d'architecture | 0,8 s | 1,5 s | 3 s |
| Meilleur débit (décisions par minute) | 108 (2 en parallèle) | 78 (4 en parallèle) | 58 (4 en parallèle) |
| Vitesse de génération en séquentiel | 167 tokens/s | 84 tokens/s | 55 tokens/s |
| Énergie GPU par décision (séquentiel → meilleur parallèle) | 56 → 40 J | 109 → 56 J | 167 → 79 J |

Pendant le calcul, le GPU consomme environ 76 W et monte jusqu'à 80 °C (8b, 4 requêtes en parallèle).

### Ce que les mesures nous apprennent

| Constat | Conséquence |
|---|---|
| **Les latences confirment les hypothèses**, et le 8b est même plus rapide que prévu (2,1 s au lieu de 3 s). | Le dimensionnement tient. Pour les 2 155 appels estimés d'un run : environ 41 min en séquentiel, environ 25 min avec le meilleur parallélisme de chaque palier. |
| **Le parallélisme double le débit** du 4b et du 8b et divise par deux l'énergie par décision. | Argument direct pour la sobriété (E5) : regrouper les appels coûte moins cher. |
| **La bascule de palier est coûteuse** : charger le 8b décharge à la fois le 1.7b et le 4b (6,5 s), et revenir coûte environ 3 s par modèle. Un aller-retour ≈ 13 s. | Si le palier grand est appelé un jour sur deux, cela ajoute environ 10 minutes par run. L'ordonnancement par palier (étape 5) doit limiter le nombre de bascules. |
| **Les réponses sont plus longues que prévu** : environ 110 tokens au lieu de 70. | À maîtriser à l'étape 6 (justification plus courte). |
| **Seules 2 réponses sur 117 sont invalides** (1,7 %), et toutes pour la même raison : elles atteignent le plafond de 160 tokens et le JSON est coupé. Aucune réponse n'est hors schéma. | Le JSON imposé fonctionne même sur le 1.7b. À l'étape 6 : plafond relevé ou réponses raccourcies, et le mécanisme « une relance puis repli » (A4) couvrira le reste. |
| **Le 1.7b à 4 requêtes simultanées est instable** : la première vague d'appels prend 4 à 12 s avant de revenir à la normale (constaté deux fois). | Pour le palier petit, on visera 2 requêtes en parallèle ; 4 pour les paliers moyen et grand. |
| **Les modèles ont une préférence** : sur des situations variées, le 8b a répondu « tendance » 24 fois sur 38. | Point d'attention pour la question Q2 du jury : un sens partagé peut venir du modèle lui-même et non de la diffusion. Le run témoin de l'étape 14 devra le mesurer. |

> **Premier chiffrage de la sobriété (E5), à confirmer à l'étape 16.** Avec la répartition prévue (270 appels au 8b, 715 au 4b, 1 170 au 1.7b), un run consomme environ 110 kJ côté GPU, soit 0,03 kWh, et environ 25 min de calcul. Les mêmes 2 155 appels tous sur le 8b demanderaient environ 170 kJ et 37 min. Ces chiffres ne comptent que le GPU pendant les appels, sans les bascules ni le reste de la machine.

### État du dépôt après l'étape

```
llm/benchmark.py                            script de mesure, relançable
docs/mesures/benchmark_2026-10-16_1143.md   résumé des mesures
docs/mesures/benchmark_2026-10-16_1143.json  mesures brutes, appel par appel
tests/test_benchmark.py                     2 tests (prompt déterministe, taille)
```

Tests : 18 tests réussis, aucune remarque de ruff.

### À retenir pour l'oral

- Le dimensionnement repose sur des **mesures**, pas sur des hypothèses : chaque chiffre renvoie à un fichier de `docs/mesures/`, appel par appel.
- Le facteur limitant est la **mémoire graphique** : les trois modèles ne tiennent pas ensemble, et changer de palier coûte environ 13 s. C'est ce qui justifie l'ordonnancement par palier.
- La mesure de l'énergie est en place dès maintenant : le coût de chaque run (E4) pourra être comparé à son apport (E1).

### Prochaine étape

Étape 4 — Contrat d'événements : l'enveloppe commune à tous les événements (identifiant, version de schéma, `run_id`, deux horloges, `caused_by`), les schémas Pydantic de chaque type d'événement, et l'isolement des événements non conformes dans `sim.dlq`.

# Étape 4 — Contrat d'événements

16 octobre 2026 · Exigences concernées : F4 (schémas documentés et versionnés, événements non conformes isolés), C1 (granularité justifiée), C3 (deux horloges), F6 (traçabilité), B1 (sens en catégories fermées).

### Objectif

Fixer, avant d'écrire le moindre agent, la forme exacte de tout ce qui circule sur Kafka. Ce « contrat » est ce que la simulation, les indicateurs, l'historique, le rejeu et Godot ont en commun : s'il est clair et vérifié, chaque composant peut être écrit et testé séparément.

### Ce qui a été fait

- `bus/events.py` : l'enveloppe commune et les 12 types d'événements, chacun avec son schéma Pydantic, sa version et son topic.
- `sim/domaine.py` : le vocabulaire fermé du monde (7 zones, 4 sens, 7 actions) et le temps simulé.
- `bus/producer.py` : publie un événement sur le topic de son type, avec accusé de réception et sans doublon.
- `bus/consumer.py` : lit et valide chaque message ; un message non conforme est envoyé dans `sim.dlq`, compté, et la lecture continue.
- `bus/export_schemas.py` : génère la documentation du contrat (`docs/evenements/README.md` et un JSON Schema par type) directement depuis le code. Un test échoue si la documentation n'est plus à jour.

### L'enveloppe commune

Tout événement porte les mêmes champs, quel que soit son type :

| Champ | Rôle | Exigence |
|---|---|---|
| `event_id` | Identifiant unique (UUID) | F6 |
| `schema_version` | Version du schéma du type ; une version inconnue est rejetée | F4 |
| `run_id` | Run qui a produit l'événement | E2, E3 |
| `type` | Type d'événement, qui détermine la forme du `payload` | F4 |
| `sim_time_occurred` | Moment *simulé* où le fait se produit | C3 |
| `sim_time_processed` | Moment *simulé* où l'émetteur a fini de le traiter | C3 |
| `wall_time_emitted` | Moment *réel* d'émission, en UTC | C3, E4 |
| `source_agent_id` | Émetteur : un agent ou un composant | D2 |
| `zone` | Zone concernée, si elle existe | D1 |
| `caused_by` | Identifiants des événements à l'origine de celui-ci | F6, B4 |
| `payload` | Contenu propre au type | — |

> **Les deux horloges (C3).** Un agent déclenché le jour 3 à 10:00 (`sim_time_occurred`) dont la réflexion se termine à 10:20 simulées (`sim_time_processed`) produit un événement émis à 14 h 32 min 05 s, heure réelle (`wall_time_emitted`). Le contrat interdit qu'un fait soit traité avant de s'être produit. Un message porte en plus son moment de `livraison` : il n'est lisible qu'à partir de cette date, ce qui servira à détecter les violations de cohérence temporelle (C6).

### Les 12 types d'événements

| Topic | Types |
|---|---|
| `sim.world.events` | `horloge.pas` (début d'un jour), `monde.meteo`, `monde.stock`, `monde.perturbation` |
| `sim.agent.messages` | `agent.message` : direct, à toute une zone, ou sur les réseaux sociaux |
| `sim.agent.decisions` | `agent.decision` (action, sens, justification, porteur ou non, moteur, coût de l'appel LLM), `agent.deplacement` |
| `sim.indicators` | `indicateur.valeur`, avec sa fenêtre de calcul |
| `sim.control` | `controle.injection` (rupture de stock, polémique, vague de chaleur), `controle.commande` (pause, reprise, arrêt) |
| `sim.incidents` | `incident` : sortie invalide, hors rôle, sans réponse, repli, violation temporelle |
| `sim.dlq` | `dlq.rejet` : le message d'origine, son topic et la raison du rejet |

Une décision indique aussi son **moteur** : `llm`, `regle` (agent à règles) ou `repli` (agent LLM retombé sur sa règle après un échec, A4). Une décision LLM porte obligatoirement le coût de son appel (modèle, tokens en entrée et en sortie, durée, nombre de tentatives) : le coût de chaque run (E4) se calculera directement à partir des événements.

### Décisions prises à cette étape

| Décision | Raison |
|---|---|
| Temps simulé en **minutes entières** depuis le jour 1 à 00:00 | Un pas dure un jour, mais un message peut être émis le soir et lu le lendemain midi (exemple de l'énoncé pour C3). Des entiers se comparent exactement, sans erreur d'arrondi : indispensable pour rejouer un run à l'identique et détecter une violation temporelle. |
| **Un événement par fait observable** (granularité, C1) | Une décision, un message, un déplacement, un changement de météo ou de stock. Jamais un instantané complet de l'état à chaque pas : ce serait volumineux et redondant. C'est la plus petite unité qui suffit au rejeu et à la traçabilité. |
| **7 actions fermées** : acheter, appliquer, recommander, critiquer, en parler, ignorer, abandonner | Comme pour le sens, une liste fermée évite le texte libre, raccourcit les réponses des petits modèles et rend les décisions comptables. On pourra l'étendre en incrémentant la version du schéma. |
| **JSON validé par Pydantic** plutôt qu'Avro avec un registre de schémas | Lisible à l'œil, natif en Python, sans serveur supplémentaire. Les JSON Schema exportés jouent le rôle de documentation formelle. |
| Documentation **générée depuis le code** | Elle ne peut pas être fausse : un test vérifie qu'elle correspond au code. |
| Champs **obligatoires à la réception** | À la construction, l'identifiant et l'horodatage sont remplis automatiquement. Mais un message *reçu* sans identifiant est rejeté : en inventer un casserait la traçabilité. |

### Ce que le contrat refuse

En plus des types de champs, des règles de cohérence sont vérifiées à chaque validation :

- un fait traité avant de s'être produit, ou un événement qui se cite lui-même comme cause ;
- un sens, une action ou une zone hors des listes fermées, un champ inconnu ;
- une version de schéma inconnue, un horodatage sans fuseau horaire ;
- un agent à règles qui déclarerait un palier ou un appel LLM ; une décision LLM sans son coût ;
- un message livré avant d'avoir été émis, un message direct sans destinataire ;
- un déplacement qui ne change pas de zone, un stock sans zone.

### Les tests

| Test | Ce qu'il garantit |
|---|---|
| Aller-retour des 12 types | Chaque type s'encode puis se relit à l'identique, et part sur le bon topic. |
| 15 décisions invalides + 5 messages illisibles | Chaque cas de la liste ci-dessus est refusé, avec un message qui nomme le champ fautif. |
| Couverture des topics | Chacun des 7 topics transporte au moins un type ; aucun nom de type n'est en double. |
| Documentation à jour | La documentation générée correspond exactement au code. |
| Intégration : `sim.dlq` | Sur un vrai Kafka, 4 messages dont 2 invalides (texte illisible, version inconnue) : les 2 valides sont lus dans l'ordre, les 2 autres arrivent dans la file de rejet avec leur raison, et la lecture ne s'arrête pas. |

Résultat : **61 tests réussis** au total, aucune remarque de ruff.

### État du dépôt après l'étape

```
sim/domaine.py            zones, sens, actions, temps simulé
bus/events.py             enveloppe et 12 types d'événements
bus/producer.py           publication
bus/consumer.py           lecture, validation, file de rejet
bus/export_schemas.py     génération de la documentation
docs/evenements/          README.md + 12 JSON Schema (générés)
tests/test_events.py      42 tests unitaires du contrat
tests/test_bus.py         test d'intégration de sim.dlq
```

### À retenir pour l'oral

- Le contrat d'événements est le point de rencontre de tous les composants : simulation, indicateurs, historique, rejeu et Godot ne se connaissent pas, ils partagent ce contrat.
- Un message non conforme ne fait jamais tomber la simulation : il est isolé dans `sim.dlq` avec sa raison, et compté.
- `caused_by` relie chaque événement à ses causes : c'est ce qui permettra de remonter d'une valeur d'indicateur aux décisions et messages qui l'ont produite (F6, B4).

### Prochaine étape

Étape 5 — Horloge simulée et orchestrateur en mode synchrone : faire avancer le temps jour par jour, déclencher les agents, appliquer le délai de livraison des messages, identifier chaque run par un `run_id` et une graine.

# Étape 5 — Horloge simulée et orchestrateur (mode synchrone)

17 octobre 2026 · Exigences concernées : C2 (population fixée au lancement), C3 (horloge simulée explicite), C4 (mode synchrone), C6 (pas d'information du futur), A4 (un agent en panne n'arrête pas le monde), E2 et E3 (reproductibilité), F3 (état courant dans Redis).

### Objectif

Construire le moteur de la simulation : ce qui fait avancer le temps, décide quels agents réfléchissent, dans quel ordre, avec quelles informations, et publie le résultat sur Kafka. Les vrais agents LLM n'existent pas encore : on teste le moteur avec des **agents factices**, qui respectent la même interface et imitent la latence mesurée des modèles.

### Ce qui a été fait

| Fichier | Rôle |
|---|---|
| `sim/horloge.py` | Horloge simulée : elle n'avance que sur ordre de l'orchestrateur, jamais à reculons ; ordre de passage des paliers. |
| `sim/livraison.py` | Boîte aux lettres : un message n'est remis qu'une fois livré. |
| `sim/orchestrateur.py` | Le déroulé d'une journée, de l'horloge à la publication. |
| `sim/etat.py` | Écriture de l'état courant dans Redis à la fin de chaque jour. |
| `sim/run.py` | Identité du run ( `run_id` , graine, manifeste) et commande de lancement. |
| `agents/base.py` | Interface commune des agents : perception, décision, déclencheur. |
| `agents/factices.py` | Agents et météo factices, sans LLM, pour tester le moteur. |
| `agents/roles.py` | Attribution du palier selon le rôle (tableau A5 de `CLAUDE.md` ). |

### Le déroulé d'une journée

1. **00:00** : l'horloge publie le début du jour, puis les acteurs du monde publient leurs événements (météo…).
2. **10:00** : chaque agent reçoit sa **perception**, construite sur l'état de *la veille* : sa zone, la météo, la part de porteurs autour de lui, les messages qui lui ont été livrés, les perturbations survenues.
3. Seuls les agents **déclenchés** réfléchissent : premier jour, message reçu, ou perturbation. Les autres gardent leur décision précédente, sans aucun appel (règle 10).
4. Les agents à règles décident d'abord, puis les agents LLM **regroupés par palier**, avec le nombre de requêtes simultanées fixé par la configuration.
5. Publication, **dans l'ordre des identifiants d'agents** : décisions, puis déplacements et messages qui en découlent.
6. Écriture de l'état courant dans Redis.

> **La barrière synchrone.** Le jour *d*, chaque agent ne voit que l'état du jour *d-1*. L'ordre dans lequel les agents réfléchissent pendant la journée n'a donc aucun effet sur leurs décisions. C'est ce qui permet de regrouper librement les appels par modèle sans fausser le phénomène, et c'est ce qui distingue le mode synchrone du mode asynchrone (étape 15), où les agents rapides agiront avant les lents.

### Moins de bascules entre modèles

Le grand modèle ne tient pas en mémoire avec les deux autres : un aller-retour coûte environ 13 s (étape 3). L'ordre des paliers **alterne d'un jour à l'autre** : petit → moyen → grand les jours impairs, grand → moyen → petit les jours pairs. Deux jours consécutifs se touchent par le même modèle, déjà chargé.

| Ordre | Bascules sur 6 jours (mesuré par un test) |
|---|---|
| Toujours petit → moyen → grand | 11 |
| Alterné | **6**, soit une par jour |

Sur un run de 90 jours, cela représente environ 90 bascules évitées, soit une dizaine de minutes.

### Le temps et les messages

Un message émis à 10:00 avec un délai de livraison $\delta$ est lisible à partir de 10:00 + $\delta$. Les délais sont des paramètres de la configuration :

| Canal | Délai | Destinataires |
|---|---|---|
| Zone | immédiat | les agents présents dans la zone en fin de journée |
| Direct | 2 h | les agents nommés |
| Réseaux sociaux | 6 h | toute la population (provisoire, voir plus bas) |

En mode synchrone, avec la barrière, tout délai inférieur à 24 h donne une lecture le lendemain ; un délai de 30 h, le surlendemain. Les tests vérifient ces cas un par un.

### Cohérence temporelle (C6) et pannes (A4)

- **Par construction**, un agent ne reçoit que les messages déjà livrés.
- **Par contrôle**, si une décision cite ( `caused_by` ) une information que l'agent n'avait pas reçue, l'orchestrateur publie un incident `violation_temporelle`, et le compte. Un test utilise volontairement un agent « tricheur » pour le prouver.
- Un agent qui plante ou ne répond pas produit un incident `sans_reponse` et garde sa décision précédente : la journée continue.
- Une décision mal formée (message trop long…) produit un incident `sortie_invalide` et rien d'autre : jamais une décision publiée à moitié.

### Identité et reproductibilité d'un run

- Chaque run reçoit un identifiant lisible : `20261017-173539-default-s42` (date, configuration, graine) et un dossier `runs/<run_id>/` avec un **manifeste** (configuration complète, graine, versions, commit git) et un **bilan** (compteurs).
- Chaque agent a son propre générateur aléatoire, dérivé de la graine du run et de son identifiant : ses tirages ne dépendent pas de l'ordre de passage des autres.

### Deux défauts trouvés et corrigés grâce aux tests

| Défaut | Correction |
|---|---|
| Deux runs de même graine **divergeaient dès le jour 2** : les messages reçus le même jour étaient départagés par leur identifiant, tiré au hasard à chaque run. | Tri par moment de livraison puis ordre de dépôt, qui est déterministe. Un test compare maintenant deux runs lancés dans **deux processus distincts**, pour détecter aussi les dépendances à l'ordre interne de Python. |
| La lecture de `.env` **écrivait** ses valeurs dans l'environnement du processus : un test y laissait une fausse adresse Redis, et un test d'intégration lancé après était ignoré sans échouer. | Le fichier est désormais lu sans rien modifier. Un test le vérifie. |

### Ce que montrent les premiers runs

Run factice de 5 jours avec 119 agents (la météo est un acteur du monde, pas un agent) : 595 décisions, 186 messages, 157 déplacements, 0 incident, 0 violation temporelle, en moins d'une seconde. Avec les latences mesurées divisées par 10 : 11,7 s pour 3 jours, conforme au calcul (environ 3,3 s par jour).

> **Constat important pour le budget.** Aucune décision n'est jamais « conservée » : un message sur les réseaux sociaux est aujourd'hui livré à *toute* la population, si bien que chaque agent reçoit un message chaque jour et réfléchit chaque

> jour. Avec de vrais modèles, cela ferait 84 × 90 = **7 560 appels** par run (environ 50 min), au lieu des quelque 2 155 prévus. À l'étape 11, la diffusion sur les réseaux sera limitée aux **abonnés** de l'émetteur.

### Les tests

26 nouveaux tests : horloge, alternance des paliers et nombre de bascules, limite de parallélisme, ordre de publication indépendant des latences, délais de livraison (4 cas), barrière synchrone, déclencheurs, violation temporelle, citation autorisée d'un message reçu, agent en panne, décision mal formée, reproductibilité (même processus et deux processus), population factice, configuration invalide, et un run complet de 3 jours sur le vrai Kafka et le vrai Redis.

Résultat : **87 tests réussis**, aucun ignoré, aucune remarque de ruff.

## À retenir pour l'oral

- Le temps simulé est un objet explicite, piloté par l'orchestrateur : il ne dépend jamais de la vitesse des modèles en mode synchrone.
- La barrière synchrone rend l'ordre de passage sans effet, ce qui autorise le regroupement par palier et divise par deux les bascules de modèle.
- Un run est reproductible : même configuration et même graine donnent les mêmes décisions, ce qui est vérifié automatiquement.
- Les déclencheurs sont le premier levier de sobriété : sans eux, le coût d'un run triple.

### Prochaine étape

Étape 6 — Couche LLM : client Ollama avec sortie JSON imposée et réflexion désactivée, routage vers le bon modèle selon le palier, une relance puis repli sur une règle en cas de réponse invalide, incidents publiés, coût de chaque appel journalisé.

# Étape 6 — Couche LLM

18 octobre 2026 · Exigences concernées : A4 (tolérance aux pannes d'agent), A5 (modèle selon le rôle), B1 (sens en catégories fermées), E4 (coût de chaque appel), F6 (traçabilité), règle 5 de `CLAUDE.md` (sortie structurée).

### Objectif

Faire décider de vrais agents par les modèles Qwen3 : leur donner un prompt construit à partir de leur perception, obtenir une réponse dont la forme est garantie, savoir quoi faire quand le modèle échoue, et mesurer ce que coûte chaque décision. Les agents LLM sont prêts ; ils seront branchés dans un vrai run à l'étape 7.

### Ce qui a été fait

| Fichier | Rôle |
|---|---|
| `llm/sortie.py` | Schéma JSON imposé au modèle et lecture validée de sa réponse. |
| `llm/client.py` | Appel à Ollama : JSON imposé, réflexion désactivée, graine, mesure des tokens et de la durée. |
| `llm/routeur.py` | Quel modèle pour quel palier, délai maximal, préchargement du modèle avant chaque groupe. |
| `agents/agent_llm.py` | L'agent LLM : prompt, appel, une relance, puis repli sur règle. |
| `agents/prompts.py` | Prompts (rôle, produit, situation du jour), construits depuis la configuration. |
| `agents/regles.py` | Seuil d'adoption individuel et règle de repli. |
| `llm/validation.py` | Mesure de la couche sur les vrais modèles ( `python -m llm.validation` ). |
| `sim/orchestrateur.py` | Publie les incidents de chaque décision et totalise les coûts par modèle. |

### Décisions prises à cette étape

| Question | Décision | Raison |
|---|---|---|
| Règle de repli quand le modèle échoue deux fois | **Seuil + continuité** : l'agent garde son avis de la veille ; s'il n'en a pas, il adopte si la part de porteurs autour de lui atteint son seuil individuel | Un repli ne doit pas changer brutalement le sens d'un agent. La règle de seuil est celle de Granovetter, réutilisée par les agents à règles (étape 13). |
| Ce que le modèle décide dans sa réponse | Justification, sens, action, et un message facultatif avec son canal (zone, direct à un contact, réseaux) | Le canal fait partie de la décision de parler. Les déplacements restent gérés par des règles (étape 11) : un champ de plus coûterait des tokens et des erreurs au petit modèle. |
| Porteur ou non | Déduit de l'action : acheter ou appliquer fait un porteur, abandonner l'en retire, le reste ne change rien | Une règle simple et explicable, plutôt qu'un champ de plus à remplir par le modèle. |
| Modèle de chaque palier | Dans la configuration ( `llm.modeles` ) | Les configurations C2 (tout sur le 8b) et C3 (tout sur le 1.7b) se lancent sans toucher au code. |
| Réglages des appels | Sortie plafonnée à 320 tokens, délai maximal 60 s, température 0,8 | À 160 tokens, l'étape 3 voyait des JSON tronqués. 60 s laisse de la marge au 8b chargé à froid. |

### Une réponse dont la forme est garantie

Le schéma JSON est donné au paramètre `format` d'Ollama, qui **contraint la génération** : le modèle ne peut écrire qu'un JSON de cette forme. Le sens et l'action ne peuvent prendre que les valeurs des catégories fermées (B1). Le schéma est propre à chaque agent : un message direct ne peut viser qu'un de *ses* contacts.

```
{ "justification": "…une ou deux phrases…",
  "sens": "efficace | tendance | banal | depasse",
  "action": "acheter | appliquer | recommander | critiquer | en_parler | ignorer | abandonner",
  "message": null
           | { "canal": "zone | reseaux", "contenu": "…" }
           | { "canal": "direct", "destinataire": "<un contact>", "contenu": "…" } }
```

La réponse est ensuite **revalidée** par Pydantic, car la contrainte d'Ollama ne couvre pas tout (texte vide, champ en trop…).

> **Pourquoi la justification vient en premier.** Ollama écrit les champs dans l'ordre du schéma. En plaçant la justification avant le sens et l'action, le modèle formule sa raison *avant* de choisir, au lieu de justifier après coup un choix déjà fait.

Un essai préalable a montré que la longueur maximale imposée par le schéma **coupe le texte en plein mot**. Elle ne sert donc que de garde-fou (300 caractères) ; c'est le prompt qui demande des réponses courtes.

### Quand le modèle échoue : relance, puis repli (A4)

1. Réponse invalide ou délai dépassé : un incident (`sortie_invalide` ou `sans_reponse`), puis **une relance**, qui rappelle au modèle la raison du refus.
2. Second échec : incident `repli`, et la décision vient de la règle « seuil + continuité ». Elle est publiée avec `moteur = repli` : on sait toujours quelles décisions ne viennent pas du modèle.

L'agent ne publie rien lui-même : il rend ses incidents avec sa décision, et l'orchestrateur les publie sur `sim.incidents`, rattachés à la décision par `caused_by` (F6). Une décision cite dans `caused_by` exactement les messages montrés au modèle : au plus les 8 plus récents, pour tenir dans le contexte de 2048 tokens.

### Mesurer le coût (E4)

- Chaque décision porte son coût : modèle, tokens en entrée et en sortie, durée, nombre de tentatives. Les tentatives ratées sont comptées : elles ont consommé du calcul.
- Le bilan du run totalise appels, relances, replis, tokens et durée, **par modèle**.
- Avant chaque groupe de palier, le routeur **précharge le modèle** : le temps de bascule est mesuré à part au lieu d'alourdir la latence du premier agent.

### Résultats sur les vrais modèles

72 décisions (24 par modèle, un agent par rôle, situations variées), avec le parallélisme de la configuration. Détail dans `docs/mesures/couche-llm_2026-10-18_1131.md`.

| Modèle | Valides au 1er coup | Replis | Latence médiane | p90 | Tokens entrée / sortie | Messages envoyés |
|---|---|---|---|---|---|---|
| qwen3:1.7b | 24/24 | 0 | 0,9 s | 1,0 s | 581 / 105 | 0/24 |
| qwen3:4b | 24/24 | 0 | 2,5 s | 4,0 s | 580 / 117 | 7/24 |
| qwen3:8b | 24/24 | 0 | 4,0 s | 7,5 s | 592 / 119 | 24/24 |

Chargement du 8b avant son groupe : 7,4 s. Aucun texte hors français dans cette série (un essai préalable du 1.7b avait laissé passer un caractère chinois : la mesure le compte désormais).

> **Constats à garder pour la suite.**
>
> - **La propension à parler dépend de la taille du modèle** : le 8b envoie un message à chaque décision, le 1.7b jamais. C'est un effet du modèle, pas du rôle : il pèse sur la diffusion et sur la comparaison demandée par A5. À traiter à l'étape 11 (prompts par rôle, budget de messages) et à mesurer à l'étape 14.
> - Le 1.7b produit parfois une action incohérente avec son sens (« banal » mais « acheter ») et des justifications de trois phrases. Ce sera l'objet du taux de sortie de rôle (étape 14).
> - La graine rend chaque appel reproductible pris isolément, mais le traitement en lot d'Ollama peut faire varier une réponse. Le rejeu exact d'un run passera donc par son journal (étape 8), jamais par un nouvel appel au modèle.

> Ces constats, comme celui de l'étape 5 sur la diffusion « réseaux », sont consignés avec leur étape de correction dans `docs/contexte-projet.md` (section 9, « Constats ouverts »), relue au début de chaque étape.

### Les tests

39 nouveaux tests, qui tournent sans Ollama grâce à un client scripté :

- **Schéma** : catégories fermées, destinataire imposé, 8 réponses invalides refusées.
- **Routeur** : modèle de chaque palier, configuration C2, délai dépassé.
- **Graine** : stable entre deux processus.
- **Agent LLM** : réponse valide, relance avec la raison du refus, repli après deux échecs, délai dépassé puis succès, modèle injoignable.
- **Règles** : règle de repli, seuil borné et reproductible, porteur selon l'action.
- **Prompts** : marque lue dans la configuration, limite des messages montrés.
- **Orchestrateur** : publication des incidents et bilan des coûts.

Trois tests d'intégration ( `-m ollama` ) font décider un agent par palier sur les vrais modèles.

Résultat : 126 tests réussis, aucun ignoré, aucune remarque de ruff.

### À retenir pour l'oral

- La forme de la réponse est garantie par une contrainte de génération, puis revalidée : le sens ne peut pas sortir des quatre catégories.
- Un modèle qui échoue ne bloque jamais le monde : une relance, puis une règle. Chaque incident est publié, compté et rattaché à sa décision.
- Chaque décision connaît son coût, tentatives ratées comprises ; le bilan le donne par modèle.
- Changer le modèle d'un palier est un simple paramètre : c'est ce qui rend les expériences C2 et C3 possibles.

### Prochaine étape

Étape 7 — Tranche verticale : 3 agents LLM (un par palier) et 1 agent à règles sur 5 jours, de bout en bout par Kafka et Redis, lus en direct par un client console.

# Étape 7 — Première tranche verticale

18 octobre 2026 · Exigences concernées : A2 (échanges entre agents), A4, A5, C2 (population fixée au lancement), C3 (deux horloges), E4 (coûts), F3 (Kafka et Redis).

### Objectif

Faire tourner pour la première fois toute la chaîne avec de vrais modèles : orchestrateur, agents LLM des trois paliers, un agent à règles, Kafka, Redis, et un **client console** qui affiche en direct ce qui passe sur le bus. C'est la preuve que les briques des étapes 2 à 6 s'emboîtent.

### Ce qui a été fait

| Fichier | Rôle |
|---|---|
| `agents/population.py` | Construit la population depuis la configuration : agents LLM pour les rôles LLM, agent à seuil pour les rôles à règles, météo. |
| `agents/regles.py` | Ajout de l'agent à seuil, provisoire jusqu'aux vrais agents à règles (étape 13). |
| `bus/console.py` | Client console : lit les 7 topics et affiche chaque événement en une ligne lisible. |
| `sim/run.py` | Le run utilise désormais les agents LLM ; vérifie Ollama et les modèles avant de démarrer. |
| `configs/tranche.yaml` | Configuration de la tranche : 4 agents et la météo, 5 jours. |

### La population de la tranche

| Agent | Moteur |
|---|---|
| `pharmacien_01` | qwen3:8b (grand) |
| `createur_contenu_01` | qwen3:4b (moyen) |
| `baigneur_occasionnel_01` | qwen3:1.7b (petit) |
| `fidele_autre_marque_01` | règle de seuil (seuil tiré : 67 %) |
| `meteo` | acteur du monde (factice jusqu'à l'étape 13) |

Agents factices et agents réels partagent la même fonction d'identité (zone de départ, contacts, générateur aléatoire) : un run factice et un run LLM de même graine décrivent le même monde, seul le moteur de décision change. Un test le vérifie.

### Comment le lancer

```
# terminal 1 : le client console attend le prochain run
.venv\Scripts\python -m bus.console

# terminal 2 : le run
.venv\Scripts\python -m sim.run --config configs/tranche.yaml
```

Extrait de ce qu'affiche la console :

```
──────── Jour 2 ────────
J2 · 00:00  météo       nuageux, 25.7 °C, UV 8
J2 · 10:00  décision    baigneur_occasionnel_01 (qwen3:1.7b, 0.7 s, 87 tok)
                  efficace · appliquer · porteur — « … Le pharmacien mentionne que ce
                  pourrait être efficace, mais il faut être prudent. Je vais peut-être
                  l'appliquer pour voir. »
J2 · 10:00  décision    fidele_autre_marque_01 [règle]
                  banal · ignorer · non porteur — « Règle de seuil : 0% de porteurs autour
                  de lui, sous son seuil de 67%. »
```

La console est un **observateur** : elle n'écrit jamais sur le bus, et un message non conforme y est signalé sans être renvoyé dans `sim.dlq`. Sinon, chaque console ouverte dupliquerait les rejets. Elle lit sans groupe de consommateurs Kafka, ce qui ne gêne aucun autre lecteur. La sortie complète du run est conservée dans `docs/mesures/tranche_2026-10-18_1142.md`.

### Résultats du run de 5 jours

| Mesure | Valeur |
|---|---|
| Décisions | 16, dont 4 conservées sans appel (règle 10) |
| Appels LLM | 11 (8b : 3, 4b : 3, 1.7b : 5), 0 relance, 0 repli, 0 incident |
| Messages | 6, dont 5 sur les réseaux |
| Tokens | 6 445 en entrée, 1 352 en sortie |
| Temps | 64,6 s au total : 18,0 s d'appels, **46,6 s de chargement de modèles** (5 bascules) |

On voit l'influence circuler : le pharmacien publie le jour 1 que la crème « tiendra peut-être la route » ; le jour 2, le baigneur cite « le pharmacien » et passe à *efficace* et *appliquer*. Le fidèle, seul dans sa zone sans porteur, ne bouge pas.

> **Constats de la tranche**, ajoutés à la table des constats ouverts ( `docs/contexte-projet.md` , section 9) :
>
> - **Observations inventées.** Le jour 1, le pharmacien dit avoir « vu createur_contenu_01 parler de la marque sur les réseaux » alors qu'il n'a encore rien reçu. Ce n'est pas une violation de `caused_by` (rien n'y est cité), mais c'est une information du futur *dans le texte*. Étape 14 : détecter automatiquement un identifiant d'agent cité sans message reçu de lui, et le compter.
> - **Le temps de chargement domine** (72 % du run). Une bascule par jour, comme prévu à l'étape 5, mais d'environ 9 s : sur 90 jours, environ 14 min. À mesurer sur un run complet (étape 16).
> - **Personnages encore flous.** Le pharmacien parle comme un particulier (« c'est pas mon truc ») et reste *tendance* tous les jours. Un message du créateur a été coupé net par la longueur maximale. Prompts par rôle à l'étape 11.
> - **Météo factice incohérente** (« canicule » à UV 6, « nuageux » à UV 8) : vraie météo à règles à l'étape 13.

### Une remarque technique

Au premier lancement, la console a attendu quelques secondes : c'était le tout premier lecteur à se connecter au broker avec un groupe, et Kafka créait son topic interne de suivi ( `__consumer_offsets` ). La console lit désormais sans groupe, ce qui supprime cette dépendance.

### Les tests

4 nouveaux tests :

- la configuration de la tranche (un agent LLM par palier et un agent à règles) ;
- la population complète (bon moteur par rôle, seuils bornés, reproductible, mêmes identités que les agents factices) ;
- l'affichage de chaque type d'événement par la console ;
- un run de 2 jours **de bout en bout** sur les vrais Ollama et Kafka. Il vérifie que tout ce qui est publié est relu, que les appels relus égalent ceux du bilan, et que chaque décision vient du modèle de son palier.

Résultat : 130 tests réussis, aucun ignoré, aucune remarque de ruff.

### À retenir pour l'oral

- La chaîne complète fonctionne : de l'horloge aux modèles, au bus, à l'état, à l'écran.
- Le même monde peut tourner avec des agents factices ou des agents LLM : on teste le moteur sans GPU, et on compare ensuite.
- Le coût d'un run se lit dans son bilan, par modèle ; ici, le chargement des modèles coûte plus que les appels eux-mêmes.
- Les premiers défauts de comportement des agents sont visibles et consignés, avec l'étape où ils seront traités.

### Prochaine étape

Étape 8 — Historique et rejeu : écrire chaque run en Parquet, puis le relire à l'identique sans aucun appel LLM, avec les mêmes événements (E3, F5).

# Étape 8 — Historique et rejeu sans LLM

18 octobre 2026 · Exigences concernées : E3 (rejeu à l'identique, sans appel LLM), F5 (historique interrogeable par des requêtes reproductibles), F3 (où vit chaque donnée), F6 (traçabilité).

### Objectif

Garder chaque run en entier, pouvoir le **rejouer à l'identique sans aucun appel LLM** (même affichage, mêmes indicateurs) et interroger l'historique de tous les runs pour comparer des scénarios. C'est aussi le filet de la soutenance : si les modèles ne répondent pas le jour J, on rejoue un run enregistré.

### Décisions prises à cette étape

| Question | Décision | Raison |
|---|---|---|
| Qui écrit l'historique | Un **lecteur Kafka** lancé automatiquement par le run | On enregistre exactement ce qui a circulé sur le bus, pas ce que l'orchestrateur croit avoir publié. Une commande à part reconstruit l'historique depuis Kafka (rétention de 7 jours). |
| Par où passe le rejeu | **Directement depuis Parquet** vers un afficheur (console aujourd'hui, pont vers Godot à l'étape 9) | Les événements sont rendus à l'octet près et ne se mélangent jamais avec un run en direct. |
| Format | Parquet par lots ( `runs/<run_id>/evenements/part-NNNNN.parquet` ), lus avec DuckDB | Un arrêt brutal ne perd que le lot en cours. DuckDB lit tous les fichiers de tous les runs d'un coup, sans base à alimenter. |
| Ordre exact entre topics | Deux en-têtes Kafka posés par le producteur : `producteur` et `seq` (numéro de publication) | Chaque topic garde son ordre, mais pas l'ordre entre topics. L'horodatage est en millisecondes et ne départage pas des événements publiés dans la même milliseconde. Ces en-têtes ne changent pas le contrat d'événements. |

### Ce qui a été fait

| Fichier | Rôle |
|---|---|
| `replay/ecrivain.py` | Écrivain d'historique : lit les 7 topics et écrit les événements du run en Parquet. |
| `replay/rejeu.py` | Relit un run dans l'ordre de publication, à la vitesse choisie, et vérifie son bilan. |
| `replay/bilan.py` | Recalcule le bilan d'un run à partir de ses seuls événements. |
| `replay/historique.py` | Vues DuckDB sur tous les runs et requêtes nommées. |
| `bus/producer.py` | En-têtes d'ordre de publication, et clé de tri partagée par la console et le rejeu. |
| `sim/run.py` | Démarre l'écrivain avant le premier événement et attend qu'il ait tout reçu. |

### Ce que contient l'historique

Une ligne par événement. La colonne `brut` garde l'événement exactement tel qu'il a circulé, et c'est elle qui sert au rejeu. Les autres colonnes reprennent l'enveloppe (type, dates simulées, jour, agent, zone, `caused_by`) et la position Kafka (topic, offset, horodatage, producteur, numéro de publication), pour les requêtes. En fin de run, le bilan indique combien d'événements ont été publiés et combien ont été enregistrés : les deux nombres doivent être égaux.

### Le rejeu (E3)

```
.venv\Scripts\python -m replay.rejeu <run_id>              # tout, sans attente
.venv\Scripts\python -m replay.rejeu <run_id> --vitesse 2  # 2 jours simulés par seconde
.venv\Scripts\python -m replay.rejeu <run_id> --verifier  # bilan recalculé = bilan.json ?
```

Le rejeu affiche les événements avec la même fonction que la console en direct. Pour le contrôle `--verifier`, le bilan est recalculé avec la même classe `Bilan` que l'orchestrateur et dans le même ordre, si bien que même les sommes de durées tombent juste au millième près. Seuls les temps de la machine (durée totale, chargements, bascules) ne sont pas comparés : ils décrivent l'ordinateur, pas le monde simulé.

### La preuve sur un vrai run LLM

Tranche verticale relancée ( `20261018-121942-tranche-s42` ), avec la console ouverte pendant le run, puis rejouée :

| Vérification | Résultat |
|---|---|
| Événements publiés / enregistrés | 18 / 18 |
| Affichage en direct et affichage du rejeu | **30 lignes identiques** |
| Bilan recalculé depuis l'historique | **identique** à `bilan.json` |
| Appels LLM pendant le rejeu | 0 |

> **Pourquoi le rejeu doit passer par le journal.** Ce run a la même graine que celui de l'étape 7, et il est pourtant différent : 7 décisions et 1 message, contre 16 et 6. Le pharmacien (8b) a donné mot pour mot la même réponse le jour 1, mais pas le baigneur ni le créateur. C'est ce que l'étape 6 annonçait : une graine ne rend pas un run LLM rejouable. On le rejoue donc depuis son enregistrement, jamais en rappelant les modèles. Ce constat est réglé et retiré de la table.

### Deux défauts trouvés en vérifiant

- **Une course à la fin du run** : quand l'écrivain avait déjà tout reçu, la fin de run rendait la main avant l'écriture du dernier lot, et le bilan annonçait 0 événement enregistré. La fin de run attend maintenant que le dernier fichier soit écrit.
- **La console en direct n'affichait pas tout à fait l'ordre réel.** En reconstruisant l'historique de la tranche de l'étape 7 depuis Kafka, une ligne du rejeu n'était pas au même endroit qu'à l'écran. C'est la console qui avait tort : elle triait par heure de création des événements, et non par ordre de publication. Elle utilise désormais la même clé de tri que le rejeu.

### L'historique interrogeable (F5)

Les vues `evenements`, `runs`, `decisions`, `messages` et `incidents` lisent directement les fichiers. Les champs des décisions (sens, action, modèle, tokens…) sont extraits du JSON brut, donc exactement de ce qui a circulé. Cinq requêtes nommées servent au rapport : `runs`, `sens_par_jour`, `cout_par_modele`, `incidents_par_nature`, `messages_par_palier`.

```
.venv\Scripts\python -m replay.historique --requete cout_par_modele
.venv\Scripts\python -m replay.historique --sql "SELECT palier, count(*) FROM decisions GROUP BY 1"
```

La requête `messages_par_palier` servira à mesurer le constat de l'étape 6 (propension à parler selon la taille du modèle).

### Nouveau constat

> **Le monde se fige faute de déclencheur.** Dans ce run, à partir du jour 3, plus aucun agent ne décide : sans message reçu ni perturbation, chacun garde sa décision (règle 10), même quand la météo change. Le tableau des acteurs prévoit d'autres déclencheurs (exposition à un porteur, tendance perçue, publication planifiée, bilan hebdomadaire). À traiter aux étapes 11 et 13, puis à surveiller dans les bilans (part de décisions conservées).

### Les tests

9 nouveaux tests :

- le bilan reconstruit depuis les seuls événements est égal au bilan tenu en direct, sur un run qui comporte relances, replis, un agent en panne et une décision mal formée ;
- l'historique Parquet redonne les événements à l'octet près et dans l'ordre de publication, même quand ils arrivent topic par topic dans la même milliseconde ;

- un bilan altéré est détecté, et un historique absent est signalé ;
- la vitesse du rejeu est respectée, et l'ordre de publication est bien départagé ;
- les requêtes DuckDB sur deux runs donnent les bons totaux, sont reproductibles, et aucun incident ne cite une décision absente (F6) ; elles fonctionnent aussi sans aucun historique ;
- de bout en bout sur Kafka : tout ce qui est publié est enregistré, et le rejeu redonne les mêmes événements et le même bilan.

Résultat : **139 tests réussis**, aucun ignoré, aucune remarque de ruff. Dépendances ajoutées : `pyarrow 26.0.0` et `duckdb 1.5.6`.

### À retenir pour l'oral

- Un run se rejoue à l'identique sans appel LLM : même affichage ligne à ligne, même bilan, vérifiés automatiquement.
- Ce qui est enregistré, c'est ce qui a circulé sur le bus : l'écrivain est un consommateur Kafka comme les autres.
- Une graine ne suffit pas à reproduire un run LLM : la preuve est dans deux runs de même graine. Le journal est la seule source de vérité.
- Tous les runs s'interrogent ensemble en SQL, avec des requêtes nommées pour le rapport.

### Prochaine étape

Étape 9 — Pont FastAPI : diffuser les événements vers Godot par WebSocket, en direct ou en rejeu, inspecter un agent (D2), injecter un événement (D3), mettre en pause ou arrêter un run.

# Étape 9 — Pont FastAPI

18 octobre 2026 · Exigences concernées : D1 (affichage en direct), D2 (inspection d'un agent), D3 (injection d'un événement en direct), E3 (rejeu), C3 et C6 (dates et cohérence temporelle), F6 (traçabilité).

### Objectif

Ouvrir la simulation à une interface. Godot (étape 10) n'a pas de client Kafka : le pont reçoit les événements et les pousse par WebSocket, en direct ou en rejeu. Il répond aussi aux questions de l'observateur (qui est cet agent, pourquoi a-t-il décidé cela ?) et transmet ses actions (injecter une perturbation, mettre en pause, arrêter).

### Décisions prises à cette étape

| Question | Décision | Raison |
|---|---|---|
| Où tourne le pont | Dans **son propre processus** (`python -m bridge`) | Il ne parle à la simulation qu'à travers Kafka, Redis et Parquet. Godot reste connecté pendant que les runs démarrent et s'arrêtent. |
| Quand une injection prend effet | Au **début du jour simulé suivant** | Cohérent avec la barrière synchrone et C6 : aucun agent ne découvre une perturbation au milieu d'une journée déjà commencée. |
| Qui la perçoit | Tout le monde, ou **une seule zone** si elle est précisée | Une rupture de stock à la pharmacie ne concerne pas la plage. |
| Quelles perturbations | Les 3 existantes (rupture de stock, polémique, vague de chaleur) | Leurs effets mécaniques arrivent avec les agents à règles (étape 13) ; la liste sera enrichie à l'étape 11. |

### Ce qui a été fait

| Fichier | Rôle |
|---|---|
| `bridge/app.py` | L'application FastAPI : WebSocket, inspection, injections, commandes. |
| `bridge/vue.py` | Vue du monde construite à partir du seul flux d'événements (direct et rejeu). |
| `bridge/client.py` | Client de test du WebSocket, en attendant Godot. |
| `sim/controle.py` | Lecture de `sim.control` par l'orchestrateur. |
| `sim/orchestrateur.py` | Pause, reprise, arrêt propre, injection transformée en perturbation, ciblage par zone. |
| `sim/etat.py` | État Redis enrichi pour l'inspection, statut et heure simulée du run courant. |
| `docs/pont.md` | Référence du pont pour l'interface : routes, messages, codes de réponse. |

### Ce que propose le pont

| Route | Exigence | Rôle |
|---|---|---|
| `WS /ws` | D1 | Un instantané du monde à la connexion, puis chaque événement du run en cours. |
| `WS /ws?rejeu=<run_id>&vitesse=2` | E3 | Le même flux, relu depuis l'historique : pour Godot, rien ne change. |
| `GET /agents/{id}` | D2 | État de l'agent, justification et coût de sa dernière décision, ses 10 dernières décisions, ses messages. Pour un agent à règles : son seuil et la règle appliquée. |
| `POST /injections` | D3 | Une perturbation, éventuellement limitée à une zone. |
| `POST /commandes` | — | Pause, reprise, arrêt, appliqués entre deux jours. |
| `GET /etat`, `/agents`, `/runs` | D1, E3 | État courant, liste des agents, runs rejouables. |

FastAPI génère une page `/docs` qui permet d'essayer chaque route à la main.

### Le chemin d'une injection

1. L'observateur envoie `POST /injections`. Le pont publie un `controle.injection` sur `sim.control`, daté de l'heure simulée courante du run.
2. Avant le jour suivant, l'orchestrateur relève `sim.control` (un message non conforme part dans `sim.dlq`).
3. À 00:00 du jour suivant, il publie une `monde.perturbation` dont `caused_by` pointe sur l'injection (F6).
4. Les agents de la zone la perçoivent, ce qui les déclenche. Leurs décisions citent la perturbation, et leurs messages propagent l'effet.

### La démonstration sur un vrai run LLM

Pont, client WebSocket et tranche de 5 jours. Au jour 1, injection d'une polémique sur les réseaux sociaux : « une vidéo virale accuse la crème de laisser des traces blanches et de coller au sable ». Trace complète dans `docs/mesures/pont_2026-10-18_1345.md`.

| Jour | Ce qu'on voit dans le flux |
|---|---|
| J3, 00:00 | La perturbation est publiée pour la zone *réseaux sociaux*. |
| J3 | Le pharmacien, seul dans cette zone, passe de *tendance* à *dépassé* · *critiquer* : « les traces blanches c'est pas une option ici ». Il publie : « Trop de traces blanches, trop collante. » |
| J4 | Le baigneur passe d'*efficace* à *banal* (« le pharmacien m'a dit qu'il ne l'utiliserait pas »). Le créateur *critique* à son tour, en citant `pharmacien_01`. |

L'effet de l'injection se propage donc d'un agent à l'autre par les messages, ce que D3 demande. L'inspection du pharmacien montre sa justification, son coût (qwen3:8b, 579 tokens en entrée, 104 en sortie, 2,3 s) et une décision dont `caused_by` pointe sur la perturbation. Le bilan compte `injections : 1`, et l'injection du pont est enregistrée dans l'historique avec le reste (`evenements_externes : 1`).

### Défauts trouvés en vérifiant

- **Heure simulée d'une injection.** Dans la démonstration, l'injection est datée J1 10:00 alors que la simulation était déjà au jour 2, d'où un effet au jour 3. Le pont lisait l'heure du dernier état écrit. L'orchestrateur publie maintenant son heure courante au début de chaque jour : l'injection est datée juste (C3), et prend effet au jour qui suit.
- **Comptage de l'historique.** L'écrivain enregistre aussi, à raison, les injections et commandes publiées par le pont pendant le run. Le contrôle de fin de run compare désormais les événements de l'orchestrateur et compte à part ceux du pont.

### Nouveau constat

> **Une perturbation n'a encore qu'un effet narratif** : son texte est montré aux agents concernés, mais une rupture de stock n'empêche encore aucun achat et une vague de chaleur ne change pas la météo. Ces effets mécaniques viendront avec les agents à règles et la vraie météo (étape 13). Ajouté à la table des constats ouverts.

### Les tests

17 nouveaux tests :

- **Orchestrateur** : une injection devient une perturbation le jour suivant, dans sa seule zone, datée et traçable, et recomptée par le rejeu. Pause puis reprise ; arrêt propre, y compris pendant une pause.
- **Vue et suivi** : la vue du monde est reconstruite depuis les événements ; le suivi en direct bascule sur un nouveau run et ignore les autres ; un client trop lent est déconnecté.
- **API** : injection publiée pour le run courant et datée de son heure simulée ; demandes invalides refusées (422) ; rien n'est publié sans run en cours (409) ; commandes ; inspection d'un agent LLM et d'un agent à règles ; instantané à la connexion ; rejeu par WebSocket identique à l'historique.

- **De bout en bout** : le vrai pont, sur Kafka et Redis, injecte une perturbation dans un vrai run puis l'arrête. La perturbation de l'historique cite bien l'injection, et le run s'arrête proprement avant sa fin.

Résultat : **156 tests réussis**, aucun ignoré, aucune remarque de ruff. Dépendances ajoutées : `fastapi 0.144.0`, `uvicorn 0.54.0`, `websockets 17.2`.

### À retenir pour l'oral

- Le pont est indépendant de la simulation : il ne connaît que le bus, l'état et l'historique.
- Une injection suit le même chemin que tout le reste : un événement versionné, validé, daté dans les deux horloges et traçable jusqu'à ses effets.
- On a vu une polémique se propager : un agent la perçoit, critique, et deux autres reprennent ses mots le lendemain.
- Direct et rejeu passent par le même WebSocket : la démonstration peut se faire sur un run enregistré si les modèles font défaut.

### Prochaine étape

Étape 10 — Scène Godot 3D : plusieurs maquettes low-poly à comparer (bâtiments distincts par zone, plage avec sable et mer, personnages animés), puis la scène choisie branchée sur le pont, avec le panneau d'indicateurs et l'inspection d'un agent (D1, D2, D3).

# Étape 10 — Scène Godot 3D

18 octobre 2026 · Exigences concernées : D1 (affichage en direct, compréhensible pour un observateur), D2 (inspection d'un agent), D3 (injection), E3 (rejeu), C8 (deux tailles de population à montrer).

### Objectif

Montrer la simulation dans une scène 3D low-poly au rendu léger : une station balnéaire avec ses 7 zones et ses habitants animés, alimentée en direct (ou en rejeu) par le pont. Comme convenu au départ, le binôme a d'abord choisi une direction visuelle parmi plusieurs maquettes.

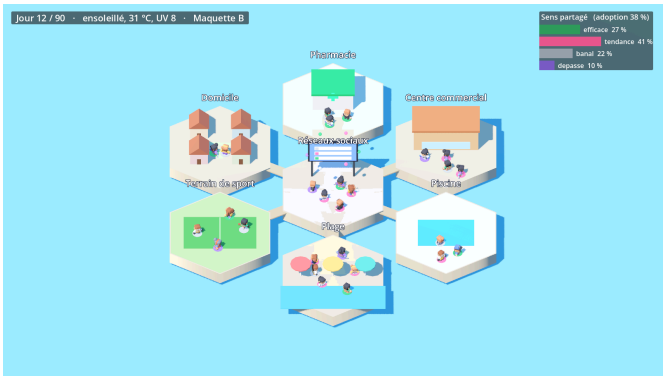
### 1. Trois maquettes, un choix

Les modèles 3D viennent de kits gratuits sous licence CC0 (Kenney, Kay Lousberg), conservés avec leur licence dans `godot/assets/` (crédits dans `godot/LICENCES.md`). Les trois maquettes montrent le même contenu : 7 zones, une trentaine d'habitants dont la pastille donne le sens, et le même bandeau d'indicateurs. On a mesuré leur coût de rendu, car Godot partage la carte graphique avec les modèles de langage.

| Maquette | Principe | Mémoire vidéo | Appels de dessin |
|---|---|---|---|
| **A — Station balnéaire** | Ville organique au bord de l'eau, modèles Kenney texturés | 211 Mo | 400 |
| B — Îlots | Une plateforme par zone autour des réseaux sociaux, formes simples | 208 Mo | 169 |
| C — Quartier | Avenues et immeubles KayKit en damier, plage au sud | 216 Mo | 373 |

Les trois tournent à 60 images par seconde (plafond de l'écran). **Le binôme a choisi la maquette A** : c'est la plus proche d'un vrai lieu, avec des bâtiments distincts par zone et une vraie plage. Les maquettes restent dans `godot/maquettes/` et `docs/maquettes/`.









### 2. La scène définitive

| Fichier | Rôle |
|---|---|
| `godot/scripts/decor.gd` | Le décor de la maquette A, les emplacements de chaque zone, la météo, l'écran des réseaux, les zones signalées. |
| `godot/scripts/habitant.gd` | Un habitant = un agent : modèle animé, pastille de sens, flacon s'il est porteur, bulle de message, marche, clic. |
| `godot/scripts/pont.gd` | Client du pont : WebSocket (avec reconnexion en direct) et requêtes HTTP. |
| `godot/scripts/interface.gd` | Bandeau, indicateurs, fil des événements, fiche d'agent, injection, commandes, choix du rejeu. |
| `godot/scripts/monde.gd` | La scène principale : applique chaque message du pont au décor et aux habitants. |
| `godot/tests/test_monde.gd` | Test de la scène sans écran ni pont. |

### Comment lire la scène (D1)

| Ce qu'on voit | Ce que ça signifie |
|---|---|
| Pastille sous les pieds | Le sens du produit pour l'agent : efficace, tendance, banal, dépassé |
| Flacon jaune au-dessus de la tête | L'agent utilise le produit (porteur) |
| Geste « oui » ou « non » | Il vient d'acheter ou d'appliquer, ou au contraire de critiquer ou d'abandonner |
| Bulle | Un message qu'il envoie (blanc : à sa zone, bleu : direct, mauve : réseaux) |
| Écran de la place | La dernière publication sur les réseaux sociaux |
| Zone surlignée en rouge | Une perturbation vient de la toucher |
| Marche | Un déplacement d'une zone à l'autre |

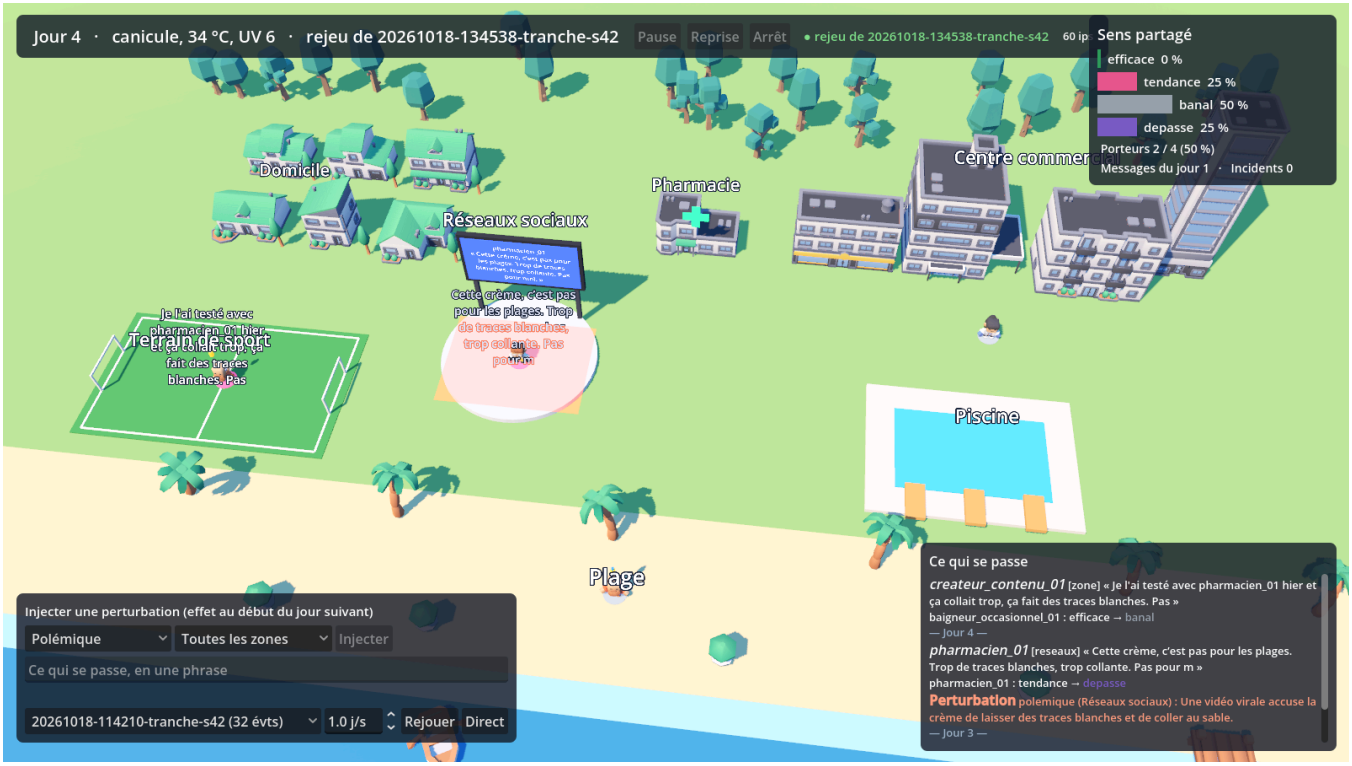
Le panneau de droite donne la répartition des sens, la part de porteurs, les messages du jour et les incidents. Le fil « Ce qui se passe » raconte les changements de sens, les messages et les perturbations. La caméra se déplace à la souris : molette pour zoomer, clic droit pour tourner, flèches pour se déplacer.

### Inspection, injection, rejeu

- **D2** : un clic sur un habitant ouvre sa fiche. On y trouve son rôle et son palier, son sens et son seuil, sa dernière décision avec sa justification, son modèle et son coût, ses messages reçus et envoyés, et ses décisions précédentes. En rejeu, la fiche est construite à partir des événements relus.
- **D3** : le panneau du bas envoie une perturbation (nature, zone, description), ainsi que pause, reprise et arrêt. Ces commandes sont désactivées en rejeu.
- **Rejeu** : on choisit un run enregistré et une vitesse en jours par seconde. Le même code affiche le direct et le rejeu.

## Vérifications

**En rejeu**, sur la tranche LLM de l'étape 9 : on voit la polémique arriver au jour 3, puis le pharmacien critiquer et publier, et sa critique s'afficher sur l'écran de la place et dans le fil.




**En direct**, avec la population complète (119 agents factices) et une vague de chaleur injectée sur la plage pendant le run : la plage est surlignée au jour suivant, et la fiche du pharmacien est complétée par l'état Redis. Le run garde un historique complet (1 529 événements publiés, 1 529 enregistrés, plus l'injection).




## Défauts trouvés et corrigés

- **La mer était cachée sous la pelouse** dans la première maquette : le sol allait jusqu'au large.

- **L'instantané du pont ne donnait pas l'identifiant des agents** : la scène ne pouvait pas savoir qui était qui à la connexion. Le pont l'ajoute maintenant, et un test le vérifie.
- **La fin d'un rejeu s'affichait comme une coupure du pont** : le dernier message arrivait juste avant la fermeture. Le client lit maintenant les messages jusqu'à la fermeture.
- **Texte de l'écran et bulles trop petits**, panneaux qui se chevauchaient : tailles et positions revues.

### Nouveau constat

> **Lisibilité à 120 agents.** Les zones les plus fréquentées deviennent des foules où les pastilles se masquent. Le panneau d'indicateurs reste juste, mais la scène ne laisse plus voir la répartition d'un coup d'œil. Piste : une jauge des sens au-dessus de chaque zone, à l'étape 14 (indicateurs par groupe), et une population de démonstration réduite à l'étape 18 (C8). Ajouté à la table des constats ouverts.

### Les tests

- `godot/tests/test_monde.gd` fait tourner la scène sans écran. Il lui donne un instantané, des décisions, un déplacement, un message sur les réseaux, un changement de jour, une perturbation et un nouveau run. Il vérifie ensuite les habitants, leur sens et leur flacon, leurs zones, la bulle, l'écran, la zone signalée et le contenu de la fiche d'inspection.
- pytest le lance (marqueur `godot`) quand le chemin de Godot est renseigné (`GODOT_BIN` dans `.env`).

Résultat : 157 tests réussis, aucun ignoré, aucune remarque de ruff.

### À retenir pour l'oral

- La direction visuelle a été choisie sur maquettes, avec leur coût de rendu mesuré.
- La scène ne contient aucune logique de simulation : elle applique les événements du pont, ce qui garantit qu'en rejeu elle montre exactement le run enregistré.
- Tout ce que l'énoncé demande à l'interface est là : voir (D1), interroger (D2) et perturber (D3), en direct comme en rejeu.
- Le rendu est léger : environ 210 Mo de mémoire vidéo, à côté des modèles de langage.

### Prochaine étape

Étape 11 — Le monde : choix du nom de marque avec un test de contamination, population complète (abonnés des réseaux, routines de déplacement, déclencheurs supplémentaires), prompts par rôle. Les constats ouverts des étapes 5 à 9 y sont traités.

# Étape 11 — Le monde : marque, population, réseau, déclencheurs

18 octobre 2026 · Exigences concernées : A1 (monde fictif), A2 (échanges entre agents), A5 (modèle selon le rôle), C2 (population au lancement), E4 et E5 (coût et sobriété), F6 (traçabilité). Cette étape règle la plupart des constats ouverts des étapes 5 à 9.

### Objectif

Donner au monde sa forme définitive : le nom de la marque, une population complète avec un réseau social et des habitudes de déplacement, des agents qui reconsidèrent leur avis quand il se passe quelque chose (et seulement alors), et des personnages qui parlent selon leur rôle. Le binôme veut une simulation aussi dynamique et réaliste que possible, avec un coût maîtrisé.

### 1. Le nom de la marque, choisi par un test de contamination

Un nom que les modèles croient connaître les ferait raisonner avec leurs connaissances réelles. Le module `llm/contamination.py` soumet chaque nom candidat, sans aucun contexte, aux trois modèles (2 questions × 4 essais chacun) et compte les réponses où le modèle prétend le connaître ou cite une marque réelle. Les meilleurs candidats ont ensuite été cherchés sur le web.

| Nom | Prétend connaître | Sur le web | Décision |
|---|---|---|---|
| **Vaelys** | 0 % (les 3 modèles) | aucune marque | **retenu** |
| Zelvane | 8 % (le 4b invente « une entreprise française ») | aucune marque | écarté |
| Nérilia | 4 % | trop proche de « néroli » | écarté |
| Ostrane, Aubelys | 29 à 33 % | — | écartés (« film de 1963 de Méliès »…) |
| **nacrelle** (ingrédient) | 0 % | inexistant (proche de « nacre ») | **retenu** |

L'indice est **Halo 4, sur une échelle inventée de 1 à 5** : un « SPF 50 » ferait raisonner les modèles avec les vrais indices. Le 8b ne prétend jamais connaître un nom ; c'est le 4b qui invente le plus volontiers. Résultats : `docs/mesures/contamination_2026-10-18_*.md`.

### 2. Un monde qui vit

| Élément | Ce qui a été fait | Constat réglé |
|---|---|---|
| **Réseau social** | Chaque agent suit 6 comptes, tirés selon l'audience de leur rôle (un créateur de contenu est dix fois plus suivi qu'un baigneur). Une publication ne touche que les abonnés de son auteur. Chacun a aussi 4 proches pour les messages directs, en partie réciproques. | Diffusion à toute la population (étape 5) |
| **Routines** | Chaque matin à 08:00, chacun gagne sa zone du jour selon son rôle et le temps (le baigneur à la plage s'il fait beau, chez lui s'il pleut ; le pharmacien à la pharmacie). Le déplacement cite la météo comme cause. | Monde figé (étape 8) |
| **Déclencheurs** | En plus d'un message ou d'une perturbation : *exposition* (la part de porteurs autour de lui a changé de 15 points), *météo marquée* (début de canicule ou d'UV fort), *lassitude* (10 jours sans y repenser). Chaque décision publie son `motif`. | Monde figé (étape 8) |
| **Budget de parole** | Messages par semaine selon le rôle (créateur 5, responsable de marque 4, pharmacien 3… baigneur et sceptique 1). Budget épuisé : le schéma JSON n'autorise plus que `message: null`. | Parole selon la taille du modèle (étape 6) |
| **Prompts par rôle** | Expertise, ton et habitudes de parole de chaque rôle ; consigne « n'invente ni message, ni personne, ni chiffre » ; une phrase de moins de 200 caractères ; le motif du jour. | Personnages flous (étape 7) |
| **Perturbations** | 4 natures de plus : promotion concurrente, article de presse, publication virale, orage. | Liste enrichie (étape 9) |

Le contrat d'événements passe en **version 1.1** pour `agent.decision` (motif), `monde.perturbation` et `controle.injection` (nouvelles natures). Les historiques déjà enregistrés en 1.0 restent lisibles : une version n'est jamais retirée.

### 3. Configurations

Une configuration peut maintenant **hériter** d'une autre (`herite_de: default.yaml`) et ne préciser que ce qui change. Décision du binôme :

- `default.yaml` : la référence, 120 agents sur 90 jours ;
- `dev.yaml` : 60 agents sur 60 jours, mêmes proportions, pour le développement ;
- une population réduite de soutenance sera ajoutée à l'étape 18.

### 4. Trois mesures, et une décision du binôme

Run LLM de 15 jours avec la configuration de développement (60 agents dont 42 LLM).

| Version | Appels LLM | Décisions conservées | Projection 120 × 90 | Ce qu'on a vu |
|---|---|---|---|---|
| 1. Déclencheurs seuls | 593 | 7 % | ≈ 7 000 appels | Les messages déclenchent 89 % des décisions : un créateur a 28 abonnés, la moitié de la ville |
| 2. + délai de réflexion de 3 jours | 210 | 67 % | ≈ 2 500 | Tout le monde décide ensemble tous les 3 jours, puis plus rien deux jours |
| 3. Délai tiré entre 2 et 4 jours | **227** | **65 %** | **≈ 2 500** | 13 à 28 décisions chaque jour, sans vagues |

> **Le délai de réflexion** (choisi par le binôme parmi quatre options) : un agent ne reconsidère pas plus d'une fois tous les 3 jours à cause d'un message, de l'exposition, de la météo ou de la lassitude. Les messages reçus entre-temps s'accumulent et sont tous lus à la décision suivante ; une perturbation reste immédiate. La deuxième mesure a montré un artefact : partis ensemble le jour 1, tous les agents restaient synchronisés. Le délai est donc tiré à chaque décision entre 2 et 4 jours (3 en moyenne) avec le générateur de l'agent : le coût ne change pas, les vagues disparaissent.

#### Résultats du run final (`20261018-174107-dev-s42`)

- 318 décisions, 227 appels (1.7b : 135, 4b : 77, 8b : 15), 0 incident, 0 relance, 0 repli ; 4 min 30 s ; rejeu fidèle.

- Motifs : première décision 60, message 225, exposition 20, météo 13.
- **La parole suit le rôle** : les habitués et les pharmaciens parlent dans leur zone (14 et 13 messages), les créateurs publient sur les réseaux (9), les baigneurs et les sceptiques se taisent presque (1). Le 8b ne parle plus à chaque décision.
- Adoption : 16 porteurs sur 60 au jour 1, 52 au jour 15 ; le sens dominant passe de « banal » à « tendance ».

> **Un phénomène émergent.** Sans que rien ne l'ait semé, une rumeur est née et a circulé : « l'air chaud est capté dans le tube ». Un créateur la publie, un pharmacien interroge un baigneur à son sujet, un habitué résume « habitue_soin_07 mentionne l'air dans le tube, pharmacien_02 est circonspect », un baigneur s'en inquiète. C'est exactement le genre de sens négocié par la conversation que la simulation doit faire apparaître (B1, B2) ; on pourra le suivre avec `caused_by` (F6).

### Nouveaux constats

- **L'adoption sature trop vite** : 52 porteurs sur 60 au jour 15, presque aucun abandon. Sur 90 jours, il faut de la place pour un pic puis un déclin : calibrage aux étapes 13 (seuils, concurrent, stocks) et 16 (comparaison à la baseline et à la courbe de Bass).
- **Le 4b confond parfois l'indice Halo et l'indice UV** (« l'indice UV 4 ») : une fiche produit claire dans le corpus (étape 12).

Ils sont ajoutés à la table des constats ouverts. Réglés et retirés : diffusion à toute la population, parole selon la taille du modèle, personnages flous, monde figé.

### Les tests

- **Configuration** : noms fictifs retenus, héritage (dev, tranche, population remplacée et non fusionnée, mêmes proportions), héritage circulaire refusé.
- **Réseau** : 6 abonnements par agent, jamais soi-même, créateurs plus suivis, reproductible ; zone de départ prise dans la routine.
- **Routines** : pas de plage par mauvais temps, reproductibles ; déplacements du matin publiés avant les décisions, avec la météo pour cause.
- **Diffusion** : une publication n'atteint que les abonnés.
- **Déclencheurs** : chaque motif, le délai de réflexion, les messages accumulés lus à la décision suivante, les décisions désynchronisées, le motif publié et recompté par le rejeu.
- **Budget et prompts** : le budget épuisé ferme le champ message puis se rouvre une semaine plus tard ; les prompts portent la marque, les habitudes de parole et la consigne contre les inventions.
- **Schémas** : un événement 1.0 d'un ancien run se lit toujours.

Résultat : 173 tests réussis, aucun ignoré, aucune remarque de ruff.

### À retenir pour l'oral

- Le nom de la marque n'a pas été choisi au goût : il a passé un test de contamination sur les trois modèles, et une recherche sur le web.
- Un agent ne réfléchit que s'il se passe quelque chose, et pas plus d'une fois tous les trois jours : c'est réaliste, et c'est ce qui borne le coût (de 7 000 à environ 2 500 appels).
- Chaque décision dit pourquoi elle a été prise (motif), ce qui permet de mesurer le coût de chaque déclencheur.
- La parole dépend du rôle, plus de la taille du modèle : la comparaison A5 n'est plus faussée.
- Une rumeur est apparue d'elle-même et a circulé : un premier signe d'émergence.

### Prochaine étape

Étape 12 — Corpus fictif et RAG : fiche produit, avis, articles et communiqués du monde Vaelys (sans allégation médicale), base ChromaDB filtrée par date simulée et par rôle, mémoire des agents, évaluation de la récupération.

# Étape 12 — Corpus fictif et RAG

18 octobre 2026 · Exigences concernées : F1 (décisions appuyées sur une base de connaissances), F2 (qualité de la récupération démontrée), A1 (mémoire propre à chaque agent), C6 (rien du futur), F6 (traçabilité), E4 (coût).

### Objectif

Avant de décider, un agent ne connaissait que sa situation du jour et les messages reçus. Il doit maintenant **retrouver** ce qu'il a pu lire sur le produit (fiche, avis, articles, communiqués) et **se souvenir** de ce qu'il a vécu les jours précédents, sans jamais accéder à une information du futur ni à un document qui ne lui est pas destiné.

### Décisions du binôme

| Question | Décision |
|---|---|
| Qui écrit le corpus | Claude rédige, le binôme relit et valide ( `corpus/README.md` ) |
| Ce que disent les documents datés | **Des faits seulement** : aucun document ne dit que Vaelys « devient la mode » ou « passe de mode ». Sinon le corpus écrirait la courbe à la place des agents, et l'émergence ne prouverait plus rien. |
| Modèle d'embeddings | Mesurer trois modèles, puis choisir sur les chiffres |
| Évaluation (F2) | Un jeu de référence automatique, plus l'annotation de 30 décisions réelles par le binôme |
| Sens « depasse » | Proposé au modèle sous le mot **demode**, traduit en interne (voir plus bas) |
| Contexte de 2 048 tokens | **Budget de prompt** plutôt qu'un contexte de 4 096, trop coûteux en mémoire vidéo |

### 1. Le corpus fictif

37 documents courts (40 à 80 mots) dans `corpus/documents/` : fiche produit et **échelle Halo expliquée** (« ce n'est pas l'indice UV »), communiqués de la marque (lancement, échantillons, format familial au jour 30, animation de plage au jour 35…), 12 avis clients sur l'expérience du produit, articles, conseils d'application, documents sur le concurrent **Zelvane** (promotion au jour 21, nouvelle formule au jour 45) et 4 notes internes réservées à un rôle (guide de l'officine pour les pharmaciens, notes aux vendeurs, plan de lancement pour la responsable de marque). Chaque document porte son jour de publication et sa visibilité ; les noms du monde y sont des gabarits ( `{marque}` …) remplacés par la configuration. Aucune allégation de santé : un test le vérifie.

### 2. Le choix du modèle d'embeddings

Un jeu de référence de 40 questions en français ( `corpus/evaluation/reference.yaml` ), dont 16 **reformulées comme un agent les écrirait** (« ça me coule dans les yeux quand je cours », « le truc nacré »), sans les mots des documents. Les 24 premières, trop proches des textes, ne départageaient pas les modèles : tous réussissaient à 100 %.

| Modèle | Réussite au rang 4 | MRR | Violations de filtre | VRAM |
|---|---|---|---|---|
| **bge-m3** (retenu) | **100 %** | **0,89** | 0 | **633 Mo** |
| qwen3-embedding:0.6b | 97 % | 0,87 | 0 | 1 948 Mo |
| nomic-embed-text (anglophone) | 87 % | 0,76 | 0 | 320 Mo |

bge-m3 est le meilleur sur les reformulations et tient à côté du 8b (5,6 Go) sans saturer la carte ; qwen3-embedding, trois fois plus gourmand, l'aurait presque saturée. Le décrochage de nomic confirme qu'un modèle anglophone ne convient pas.

### 3. Comment un agent retrouve et se souvient

1. Une question est formée à partir de sa situation : ses derniers messages, les événements du jour, sinon « que faut-il savoir sur cette crème ».

2. Dans le corpus (ChromaDB, dans le processus Python) : les 4 passages les plus proches, **publiés au plus tard ce jour-là** et **lisibles par son rôle**, en privilégiant ceux qu'il n'a pas encore lus (sans cela, la fiche produit et les questions fréquentes revenaient à chaque décision).
3. Dans ses souvenirs : les 3 plus proches, **de jours passés seulement**.
4. Le tout est ajouté au prompt (« Ce que tu as lu ou entendu dire », « Ce dont tu te souviens »), et la décision publie ses **sources** ( `agent.decision` en version 1.2).
5. Après la décision, un souvenir est enregistré : ce qu'il a reçu, ce qu'il a décidé, sa raison, ce qu'il a dit.

### 4. Deux défauts trouvés par la mesure, et corrigés

> **Le sens « depasse » était compris à l'envers.** Sur 23 décisions « dépassé », 17 allaient avec *appliquer*, *recommander* ou *acheter* : sans accent, les modèles lisaient « dépasse (mes attentes) ». Le modèle choisit désormais **demode** (« démodé : sa mode est passée, on n'en veut plus »), traduit en interne vers la même catégorie : le contrat d'événements, les indicateurs et Godot ne changent pas. Mesure suivante : 3 décisions « dépassé », toutes cohérentes (ignorer, en parler). Sans cette correction, l'indicateur de sens partagé (B1) aurait mesuré un contresens.

> **Le prompt débordait du contexte.** Le plus long atteignait 2 046 tokens sur 2 048, sans place pour la réponse : Ollama tronque alors le début, c'est-à-dire la consigne de rôle. Le prompt est maintenant construit sous un budget (3,4 à 3,6 caractères par token mesurés en français ; 3,2 retenus par prudence) : on retire d'abord les messages anciens, puis le souvenir et le passage les moins proches. Les sources publiées ne citent que ce que le modèle a vu. Mesure suivante : 1 677 tokens au plus, 79 prompts réduits sur 227.

### 5. Résultats du run final ( `20261018-183042-dev-s42` , 60 agents, 15 jours)

| Mesure | Valeur |
|---|---|
| Appels LLM | 227 (comme sans RAG), 0 relance, 0 repli |
| Tokens d'entrée par appel | 1 438 en moyenne (environ 815 sans RAG : +77 %), 1 677 au plus |
| Durée | 6 min 20 s (4 min 30 s sans RAG : +40 %) |
| Sources consultées | 1 222, dont **0 lue avant sa date, 0 hors de son rôle, 0 souvenir invalide** (contrôle sur tout le run) |
| Souvenirs de 4 jours ou plus utilisés | 213 |
| Rejeu | fidèle, bilan identique |

**La mémoire en action (A1).** Le jour 13, `sceptique_08` justifie : « Je me souviens des conseils de pharmacien_02 et des messages de createur_contenu_02 ». Son souvenir retrouvé date du jour 7, six jours plus tôt : « Messages reçus : createur_contenu_02 a dit « Vaelys, avec sa nacrelle et l'indice Halo 4, est la crème solaire idéale pour la canicule » ». **Le corpus en action (F1).** Le jour 15, un créateur écrit « l'avis client du jour 9 dit que c'est plus fluide mais coûte plus cher que Zelvane », ce que dit bien l'avis `avis-05` .

### 6. L'annotation par le binôme (F2)

`python -m rag.evaluation --annoter 20261018-183042-dev-s42` a tiré 30 décisions et exporté, pour chacune, ce que l'agent avait sous les yeux : `docs/mesures/annotation_20261018-183042-dev-s42.csv` . Le binôme remplit la colonne « pertinent (0/1) » ; `--noter` calcule alors la précision (au total, pour le corpus et pour les souvenirs), qui ira dans le rapport.

### Constats

- **Réglés** : la confusion entre l'indice Halo et l'indice UV (7 cas, puis 2 résiduels grâce à la fiche « Comprendre l'échelle Halo ») ; les justifications « méta » (8, puis 1).
- **Nouveaux**, ajoutés à la table : une **invention à caractère médical** (« l'avis client mentionne des effets secondaires », qu'aucun document ne contient) à détecter à l'étape 14 ; le **serveur d'Ollama s'arrête sous charge** une fois par run («

CUDA error »), désormais rattrapé par une relance de l'embedding, à étudier à l'étape 17 ; le **coût du RAG** (+77 % de tokens) à mettre en regard de son apport à l'étape 16.
- Toujours ouvert : l'adoption sature (54 porteurs sur 60 au jour 15).

### Les tests

- **Corpus** : fictif, noms remplacés, aucune allégation de santé, types connus, jeu de référence cohérent avec le corpus.
- **Filtres** : jamais un document du futur, notes internes lues par leur seul rôle, index réutilisé tant que le corpus ne change pas.
- **Mémoire** : souvenirs privés et des seuls jours passés ; un message reçu le jour 3 revient dans le prompt du jour 9 et dans les sources (A1).
- **Robustesse** : l'agent décide même si la récupération et la mémoire tombent en panne (deux incidents publiés) ; l'embedding est relancé après un arrêt du serveur.
- **Budget** : le prompt reste sous le budget, et les sources citées sont bien celles montrées.
- **Sens** : « demode » est traduit en « dépassé », « depasse » est refusé.
- **Contrôle d'un run** : détecte un document lu avant sa date, hors de son rôle, ou un souvenir invalide.
- **Avec Ollama** : le jeu de référence avec bge-m3, au moins 95 % de réussite et aucune violation.

Résultat : 188 tests réussis, aucun ignoré, aucune remarque de ruff. Dépendance ajoutée : `chromadb 1.5.9` ; modèle : `bge-m3`.

### À retenir pour l'oral

- Les agents décident à partir de ce qu'ils ont lu et de ce dont ils se souviennent, et chaque décision dit lesquels : on peut toujours montrer « ce qu'il avait sous les yeux ».
- Le futur est hors d'atteinte par construction (filtre de date) et par contrôle (0 écart sur 1 222 sources).
- Le modèle d'embeddings a été choisi sur mesure, avec des questions reformulées comme un agent les écrirait.
- La mesure a révélé un contresens sur une catégorie de sens, corrigé avant qu'il ne fausse les indicateurs.

### Prochaine étape

Étape 13 — Agents à règles et baseline : vraie météo saisonnière, vendeurs et stocks, marque concurrente, fidèles ; effets mécaniques des perturbations ; baseline C0 sans LLM avec la même saisonnalité (A6, E1). Les constats d'adoption trop rapide y sont traités.

# Étape 13 — Agents à règles et baseline

19 octobre 2026 · Exigences concernées : A6 (agents à règles avec les agents LLM, répartition justifiée), E1 (comparaison à une version à règles fixes du même monde), E2 (plusieurs graines, dispersion), D3 (effets réels des perturbations). Constats traités : météo incohérente (étape 7), perturbations sans effet (étape 9), adoption qui sature (étapes 11 et 12).

### Objectif

Donner au monde ses mécaniques (météo, commerce, concurrent, fidèles), et construire la **baseline C0** : le même monde, soumis aux mêmes scénarios, où les rôles LLM décident par règles. C'est la référence qui permettra de dire ce que le LLM apporte (E1). Piège identifié dès la conception : si le déclin ne s'explique que par la saison, la baseline le reproduit aussi ; il faut donc la même saisonnalité dans les deux, et chercher un déclin en cours de saison.

### Décisions du binôme

| Question | Décision |
|---|---|
| Règles de la baseline C0 | **Seuil + Bass + déclin** : imitation au seuil (Granovetter), influence externe les jours de communiqué (Bass p), pression des messages reçus (Bass q), abandon quand la mode recule (Berger et Le Mens), messages types sur le même réseau |
| Renouvellement | **Le flacon s'épuise** (12 jours) : il faut décider de racheter ou non, nouveau motif `flacon_vide` (agent.decision 1.3) |
| Stocks | **Un stock par point de vente** (pharmacie, centre commercial) |
| Fin de saison | **Pas de rentrée** : le monde reste estival jusqu'au jour 90 |

### Ce qui a été fait

| Fichier | Rôle |
|---|---|
| `sim/meteo.py` | Météo saisonnière : température qui monte puis se stabilise, temps beau ou mauvais en chaîne de Markov, conditions et UV toujours cohérents. Une vague de chaleur impose 3 jours de canicule, un orage un jour de pluie, et le bulletin cite sa cause. |
| `sim/commerce.py` | Stocks par point de vente, achats, ruptures (publiées dans la zone), réassort du vendredi commandé par les vendeurs selon les ventes de la semaine. Une rupture injectée vide réellement le rayon ; une promotion concurrente dure une semaine. |
| `sim/calendrier.py` | Calendrier de Zelvane, cohérent avec le corpus : promotion aux jours 21 à 27, nouvelle formule annoncée au jour 45. |
| `agents/regles.py` | Fidèles (seuil relevé, et plus encore pendant une promotion concurrente ; retour à leur marque si la mode recule), vendeurs, marque concurrente (publie à ses dates), agent de la baseline C0. |
| `sim/orchestrateur.py` | Les perturbations agissent avant la météo du jour ; usure des flacons ; achats dans un ordre tiré avec la graine ; stocks et ruptures publiés le soir. |
| `configs/baseline.yaml`, `baseline-dev.yaml` | La baseline C0 : héritent de default et dev, avec `agents.moteur: regles`. |
| `replay/comparaison.py` | Comparaison de runs depuis l'historique : courbes de porteurs, pic, déclin, sens final, moyenne et écart-type entre graines (E2). |

### Le flacon et le commerce

Un nouveau porteur, ou un porteur dont le flacon est fini, doit acheter : là où il se trouve si c'est un point de vente fourni, sinon au mieux fourni. En rupture, sa décision est publiée mais il ne devient pas porteur, et le lendemain son prompt lui dit « tu as voulu acheter, il n'y en avait plus ». Les achats d'un même jour passent dans un ordre tiré avec la graine : par ordre alphabétique, les mêmes agents seraient toujours servis.

> **Deux calibrages en cours d'étape.** Avec les premiers stocks, les ruptures étouffaient tout (239 achats manqués pour 112 réussis sur 60 jours). Avec un réassort fixe plus généreux, chaque flacon livré était vendu : 222 achats exactement sur 5 graines, l'offre bornait l'adoption au lieu de la dynamique sociale. Les vendeurs commandent désormais ce qu'ils ont vendu dans la semaine, avec 20 % de marge : l'offre suit la demande, et une rupture signale une demande qui s'emballe.

Un autre défaut a été corrigé : les agents de la baseline gardaient le palier de leur rôle, si bien que l'orchestrateur chargeait les modèles chaque jour pour rien. Un run C0 de 60 jours est passé de 7 min 35 s à **7 secondes**.

### La comparaison C0 contre C1 (60 agents, 30 premiers jours)

C1 : un run mixte LLM (graine 42, 445 appels, 19 min). C0 : 5 runs de la baseline (graines 42, 7, 13, 21, 99, 7 secondes chacun). Même monde, même scénario, mêmes graines de population : seule la décision change. Détail : `docs/mesures/comparaison_2026-10-19_1446.md`.

| Mesure | C0 (5 graines) | C1 (mixte LLM) |
|---|---|---|
| Porteurs au jour 1 | 3,6 ± 0,9 | **22** |
| Pic | 52 à 58, entre les jours 16 et 28 | 49 au jour 21 |
| Creux en cours de saison | faible (−3 à −6) | **49 → 38 au jour 25**, puis 45 |
| Sens en fin de période (efficace / tendance / banal / dépassé) | ~3 / 48 / 8 / 1 | **33 / 15 / 12 / 0** |
| Messages | 47 à 64 | 78 |
| Appels LLM, durée | 0, 7 s | 445, 19 min |

**Ce qu'on lit.** La baseline trace une courbe en S classique (Bass) et sature vers 58 sur 60 sans redescendre : à saturation, la mode ne recule plus autour de personne. C1 diffère sur trois points. Ses agents adoptent beaucoup plus tôt : 22 sur 42 dès la première rencontre, signe d'un enthousiasme d'« assistant » plutôt que d'habitant (nouveau constat). Il culmine plus bas et connaît un creux en cours de saison : 19 flacons achetés vers le jour 13 se finissent au jour 25, 4 ne sont pas rachetés, et la baisse visible des porteurs déclenche 4 abandons par exposition, une petite cascade de déclin. Enfin et surtout, **le sens diffère** : les agents LLM jugent majoritairement le produit « efficace » (33 sur 60), quand la baseline reste sur « tendance », que ses règles attribuent à toute imitation. Ces écarts sont des pistes, pas encore des conclusions : il faudra 5 runs C1 et les indicateurs de l'étape 14 pour les établir (E2).

### Vérifications

- Le rejeu du run C1 est fidèle (bilan identique) ; aucune source lue avant sa date ni hors de son rôle (2 420 sources contrôlées).
- 0 incident, 0 relance, 0 repli ; 1 450 événements publiés, 1 450 enregistrés.
- Le run a été suivi en direct dans Godot, où les stocks s'affichent désormais au-dessus de la pharmacie et du centre commercial (« Rupture de stock » en rouge).

### Constats

- **Réglés** : la météo incohérente (étape 7) ; les perturbations sans effet mécanique (étape 9).
- **Mis à jour** : l'adoption sature encore (C0 sans déclin, C1 avec un creux puis une remontée) : il faut des runs de 90 jours (étape 16).
- **Nouveaux** : l'adoption des agents LLM dès le premier contact (étape 14 : posture d'assistant) ; les renouvellements synchronisés par un flacon de durée fixe, qui creusent un déficit artificiel (étape 16 : tirer la durée de 10 à 14 jours, comme le délai de réflexion) ; la durée d'un run C1 (19 min pour 60 agents et 30 jours, environ 1 h 50 projetées pour 120 agents et 90 jours), à planifier pour les campagnes de l'étape 16.
- **Machine** : 16 Go de mémoire vive (15,1 Go visibles par Windows ; CLAUDE.md, qui annonçait 32 Go, est corrigé) ; un run lancé en arrière-plan par l'assistant a été interrompu par manque de mémoire. Les longs runs se lancent désormais depuis un terminal PowerShell séparé.

### Les tests

- **Météo** : cohérente (pas de canicule à UV 6), saisonnière, reproductible ; vague de chaleur et orage changent réellement le temps et sont cités en cause.
- **Commerce** : achats, repli sur le point de vente le mieux fourni, rupture annoncée une seule fois et rattachée à son niveau de stock, commande selon les ventes, rupture injectée.
- **Flacons** : un flacon fini déclenche une décision au bon jour ; un achat raté garde la décision sans le produit, et l'agent le sait le lendemain ; le prompt le lui dit.
- **Agents à règles** : fidèles plus exigeants, encore plus pendant une promotion, et retour en cas de recul ; le concurrent ne parle qu'à ses dates.
- **Baseline** : adoption par imitation et par influence externe, abandon quand la mode recule ; même monde, aucun modèle à charger ; mêmes seuils qu'en C1 à graine égale ; run reproductible et rejeu identique.

Résultat : **202 tests réussis**, aucun ignoré, aucune remarque de ruff. Un test du RAG (le premier échec venait du test de mémoire, A1) a échoué deux fois sur 8 lancements, puis n'a plus échoué en 27 lancements : il reste sous surveillance.

## À retenir pour l'oral

- La baseline n'est pas un épouvantail : c'est le même monde, avec des règles reconnues (Granovetter, Bass, Berger et Le Mens), soumis aux mêmes scénarios et aux mêmes graines.
- Les perturbations ont maintenant des effets réels : une rupture vide les rayons, une vague de chaleur fait la canicule, une promotion détourne les fidèles.
- Un flacon s'épuise : l'adoption n'est plus définitive, et le déclin devient possible en pleine saison.
- Premiers écarts entre C0 et C1 : départ plus rapide, pic plus bas, creux en cours de saison, et surtout un sens différent (« efficace » contre « tendance ») — à confirmer sur 5 runs.

## Prochaine étape

Étape 14 — Indicateurs d'émergence : formule de l'indicateur de sens partagé (décision ouverte), adoption, traçabilité vers les événements bruts (F6), run témoin sans diffusion (B3), taux de sortie de rôle et détection des inventions ; jauge des sens par zone dans Godot.

# Note — Colibrì, l'inférence depuis le SSD (étudiée et écartée)

20 octobre 2026 · Entre les étapes 13 et 14 · Exigences concernées : A5 (une seule famille, plusieurs tailles), A4 et B1 (sortie structurée), E4 (coût), C7 (limites de la machine).

### La question

On nous a suggéré Colibrì, un outil qui « fait tourner le modèle sur le SSD ». Avant d'avancer, le binôme a voulu savoir s'il pouvait servir au projet : faire tourner un modèle plus grand, ou soulager la carte graphique.

### Ce qu'est Colibrì

Un moteur d'inférence open source écrit en C (Apache 2.0), pensé pour les très grands modèles *Mixture-of-Experts* (MoE). Un tel modèle est découpé en centaines d'« experts », dont quelques-uns seulement servent à chaque token : Colibrì garde la partie commune en mémoire vive et **lit les experts sur le SSD au moment où ils sont choisis**. Il fonctionne sous Windows, avec CUDA en option, une API compatible OpenAI et jusqu'à 16 conversations en parallèle. Le projet se présente lui-même comme une plateforme de recherche.

| Modèle | Fichiers | Mémoire vive conseillée | Vitesse annoncée |
|---|---|---|---|
| Qwen3.6-35B-A3B | 23 Go | 20 Go | 6 tok/s sur processeur, 30 tok/s sur RTX 3070 |
| DeepSeek V4 Flash (284B) | 167 Go | 32 Go | environ 1 tok/s |
| GLM-5.2 (744B) | 429 Go | 24 Go | 0,05 à 0,1 tok/s sur un portable de 25 Go |

### Confronté au projet

| Critère | Constat |
|---|---|
| Nos modèles | Qwen3 1.7b, 4b et 8b sont **denses** et tiennent déjà dans la carte graphique : Colibrì ne les accélère pas. |
| Volume d'appels | Environ 2 500 décisions par run. À 30 tok/s, 4 à 5 s par décision (plus lent que le 8b) ; à 1 tok/s, environ 2 minutes : un run deviendrait impossible. |
| Mémoire vive | 16 Go ici, deux processus déjà arrêtés par manque de mémoire ; le plus petit modèle utile en conseille 20. |
| Exigence A5 | Une autre famille de modèles à côté de Qwen3 brouillerait la comparaison des tailles. |
| Sortie JSON imposée | Annoncée, mais son usage par l'API n'est pas documenté : risque direct sur A4 et B1. |
| Calendrier | Un moteur de recherche, 23 à 167 Go de poids et un nouvel adaptateur, à quelques semaines de la soutenance. |

> **Décision : Colibrì n'est pas intégré à la simulation.** Il est inscrit dans les alternatives écartées de `docs/contexte-projet.md`, avec ses raisons chiffrées, pour le rapport. Il pourrait servir hors de la boucle, pour quelques dizaines d'appels où la lenteur ne gêne pas : un juge de référence plus grand (pré-annotation du RAG ou des sorties de rôle, vérifiée par le binôme), ou un test de contamination plus exigeant. Ce sera un essai isolé, seulement si le besoin apparaît aux étapes 14 ou 16.

### Sources

- Dépôt du projet : github.com/JustVugg/colibri (vitesses et besoins cités depuis son README, non mesurés par nous).
- Colibri: giant MoE models streamed from disk ; article de Gigazine.

### À retenir pour l'oral

- Lire les poids sur le SSD ne sert que pour les modèles MoE trop grands pour la machine ; nos modèles tiennent déjà en VRAM.
- Le facteur limitant d'un run est le nombre de décisions : un modèle plus grand mais plus lent rendrait les campagnes de l'étape 16 impossibles.

# Étape 14 — Indicateurs d'émergence

24 octobre 2026 · Exigences concernées : B1 (contexte commun observable), B2 (indicateur calculé pendant le run et affiché avec la scène), B3 (distinguer l'émergence de son absence), B4 (échanges qui précèdent une variation), F6 (remonter aux événements bruts), A6 et O2 (sorties de rôle). Constats traités : incohérences et caractères étrangers du 1.7b (étape 6), observations inventées (étape 7), invention médicale (étape 12), adoption au premier contact (étape 13), lisibilité de la scène (étape 10).

## Objectif

Mesurer pendant le run ce que l'énoncé appelle le « contexte commun » : ici, le sens que chaque groupe attribue au produit, et le degré auquel ce sens est partagé. Chaque valeur doit pouvoir remonter aux décisions et aux messages qui l'ont produite, et un **run témoin sans diffusion** doit dire ce qui, dans le sens observé, vient des échanges et ce qui vient du modèle lui-même (question Q2 du jury).

## Décisions du binôme

| Question | Décision |
|---|---|
| Formule du sens partagé (20 oct.) | **Deux mesures complémentaires** : la convergence `C = 1 − H / log 4` (globale, par rôle, par zone) et l'**information mutuelle rôle × sens** normalisée par l'entropie des sens, avec un test de permutation (200 tirages reproductibles) pour sa p-valeur. La convergence seule confond un consensus avec un défaut commun ; l'information mutuelle dit si les groupes pensent encore différemment. |
| Run témoin (B3) | **Aucun message remis, et personne ne voit qui porte le produit** autour de lui (ni dans le prompt, ni dans les règles). Les agents décident encore avec la météo, les perturbations, le commerce et le corpus : ce qui converge quand même vient du modèle ( `simulation.diffusion: false` ). |
| Sorties de rôle | **Cinq signes détectés dans le texte** des décisions LLM : incohérence action/sens, personne inventée, vocabulaire de santé, autre alphabet, formule d'assistant. Heuristiques : le taux est une borne basse, vérifiée à la main sur un échantillon. |
| Adoption au premier contact | Chaque rôle dit **comment il accueille un produit nouveau** ; la première décision précise que l'agent ne l'a jamais essayé. |

### Ce qui a été fait

| Fichier | Rôle |
|---|---|
| `indicators/calcul.py` | Définitions et calcul, sans Kafka : convergence, information mutuelle et permutation, adoption. Chaque valeur cite dans `caused_by` les décisions qu'elle compte (F6). |
| `indicators/service.py` | Consommateur Kafka lancé avec le run : il remet les événements dans l'ordre exact de l'orchestrateur (en-tête `seq`) et publie les indicateurs du jour sur `sim.indicators` à chaque fin de jour (B2). |
| `indicators/derives.py` | Détection des cinq signes de dérive ; les correspondants d'un agent viennent des seuls événements (messages cités, destinataires). |
| `indicators/trace.py` | D'une valeur affichée aux décisions qui l'ont fait bouger, puis aux messages que ces agents avaient sous les yeux (F6, B4) ; `--variation` trouve le jour du plus grand écart. |
| `indicators/rapport.py` | Rapport de plusieurs runs depuis leur seul historique (un run plus ancien est recalculé avec le même code). |
| `bus/events.py` | Nouvel événement `horloge.fin_jour` 1.0 (tous les faits du jour sont publiés) ; `indicateur.valeur` 1.1 (effectif et répartition par catégorie). |
| `sim/orchestrateur.py`, `sim/run.py`, `replay/ecrivain.py` | Fin de jour publiée ; service des indicateurs démarré avant le premier événement ; l'écrivain attend aussi les indicateurs du dernier jour ; mode témoin. |
| `agents/prompts.py` | Accueil d'un produit nouveau, par rôle ; « tu ne l'as jamais essayée » à la première décision. |
| `configs/temoin*.yaml` | Témoins sans diffusion : `temoin` (default), `temoin-dev` (dev), `temoin-baseline-dev` (baseline C0). |
| `godot/scripts/` | Une **jauge des sens posée au sol devant chaque zone** (`decor.gd`) et une courbe jour par jour du sens partagé et de l'information mutuelle (`courbe.gd`), alimentées par le service, en direct comme en rejeu. |

### Les runs (60 agents, 60 jours, graine 42)

Deux runs LLM lancés depuis un terminal séparé, et deux runs de la baseline, chacun avec son témoin. Détail jour par jour : `docs/mesures/indicateurs_2026-10-24_1737.md`.

| Mesure | C0 | C0 témoin | C1 | C1 témoin |
|---|---|---|---|---|
| Adoption en fin de run | 100 % | 12 % | **83 %** | **43 %** |
| Convergence C, jour 1 → jour 60 | 0,75 → 0,79 | 0,75 → 0,68 | 0,46 → 0,25 | 0,32 → 0,28 |
| Information mutuelle, jour 1 → jour 60 (p) | 0,31 → 0,50 (0,01) | 0,31 → 0,26 (0,44) | 0,85 → 0,48 (0,005) | 0,48 → 0,43 (0,005) |
| Sens dominant en fin de run | tendance | banal | efficace | banal / efficace |
| Agents LLM porteurs dès leur 1re décision | – | – | 18 / 42 | 18 / 42 |
| Décisions LLM « en dérive » | – | – | 36 / 870 (4,1 %) | 7 / 406 (1,7 %) |
| Appels LLM, durée réelle | 0, 7 s | 0, 5 s | 870, 25 min | 406, 10 min |

### Ce qu'on lit

- **Les échanges font l'adoption.** Avec la diffusion, les porteurs doublent (83 % contre 43 % pour C1, 100 % contre 12 % pour C0). C'est le résultat le plus net de l'étape.
- **Chez les agents LLM, les échanges ne créent pas de sens commun.** En C1, la convergence baisse (0,46 → 0,25) et finit au niveau du témoin. Rôle par rôle, le sens final des agents LLM est presque le même avec ou sans échanges : pharmaciens « efficace » 3 sur 3 dans les deux runs, habitués 7 sur 8 contre 5 sur 8, baigneurs 12 sur 17 contre 11 sur 17. Le rôle explique environ la moitié de la diversité des sens dans les deux cas (information mutuelle autour de 0,45). Réponse à Q2 : **le sens des agents LLM vient du modèle, du rôle et du corpus, pas de la diffusion.**

- **La diffusion mélange d'abord les groupes, puis ils se séparent à nouveau.** En C1, l'information mutuelle tombe de 0,85 à environ 0,3 entre les jours 5 et 30 (chacun entend les autres), puis remonte vers 0,5 à 0,6 : les rôles retrouvent leur lecture du produit.
- **Le surplus de « tendance » en C1** (21 contre 5 dans le témoin) vient surtout des agents à règles : 14 des 21 sont des fidèles et des vendeurs, que leur règle fait passer à « tendance » quand ils voient des porteurs autour d'eux.
- **Le biais « tendance » du 8b** (étapes 3 et 6) ne se retrouve pas ici : les pharmaciens disent « efficace » dans les deux runs. Avec 3 pharmaciens, l'échantillon est trop petit pour conclure.

> **B3 : ce que la convergence seule ne sait pas dire.** La baseline C0 est le cas d'école d'une émergence : par imitation, tout le monde passe de « banal » à « tendance ». Son témoin reste sur « banal ». Pourtant la convergence finit presque au même niveau dans les deux (0,79 contre 0,68), parce qu'un défaut commun est aussi concentré qu'un consensus. Ce qui les distingue : l'**information mutuelle** devient significative avec la diffusion (p = 0,01) et ne l'est pas dans le témoin (p = 0,44), et le **sens dominant change** (banal → tendance) au lieu de rester celui du départ. La démonstration B3 tient donc avec les deux mesures ensemble, pas avec la convergence seule. Le binôme devra dire, avant les campagnes, s'il ajoute une mesure de déplacement du sens par rapport au jour 1 (question ouverte).

### Remonter d'une valeur aux échanges (F6, B4)

`python -m indicators.trace <run> --variation` trouve la plus forte variation du run C1 : le sens partagé tombe de 0,31 à 0,20 au jour 7. Treize décisions l'ont fait bouger, dont cinq motivées par un message ; elles remontent à deux publications « réseaux », et surtout à celle de `createur_contenu_02` au jour 4 (« Vaelys, Halo 4, nacrelle : la texture fluide qui rend la peau satinée »), entendue par cinq agents. Trois jours plus tard, après leur délai de réflexion, trois baigneurs et sceptiques passent de « banal » à « efficace » et un sceptique repasse à « banal ». Détail : `docs/mesures/trace_20261024-154113-dev-s42_sens_partage_j7_1736.md`.

### Les dérives

| Signe | C1 (870 décisions) | Témoin (406) | Lecture |
|---|---|---|---|
| Autre alphabet | 21 | 6 | 1.7b : 24 sur 755 décisions (3,2 %) ; 4b : 3 sur 434 ; 8b : aucun |
| Personne inventée | 13 | 0 | Seulement quand des noms circulent ; deux cas vérifiés à la main, tous deux justes (« mon pharmacien_02 », qui ne lui a jamais écrit) |
| Incohérence action/sens | 2 | 0 | Rare : le constat de l'étape 6 est réglé |
| Santé | 0 | 1 | « la crème pourrait avoir des effets secondaires », sans source |
| Formule d'assistant | 0 | 0 | La consigne « tu n'es pas un assistant » tient dans le texte |

La posture d'assistant se voit encore dans les choix, pas dans le ton : 18 agents LLM sur 42 deviennent porteurs dès leur première décision (22 avant la consigne d'accueil), contre environ 5 sur 60 au jour 1 dans la baseline.

### Vérifications

- 0 incident, 0 relance, 0 repli, 0 violation temporelle dans les deux runs LLM ; tous les événements et indicateurs publiés sont enregistrés (2 953 et 1 187 en C1).
- Rejeu fidèle (bilan identique) d'un run de contrôle ; les indicateurs du rejeu sont ceux du run, restitués sans recalcul.
- Le même code calcule les indicateurs en direct et après coup : deux calculs d'un même run donnent les mêmes valeurs, y compris les p-valeurs (tirages de permutation reproductibles).

### Constats

- **Réglés** : les incohérences action/sens du 1.7b (2 sur 870) et les caractères étrangers, désormais comptés à chaque jour ; les observations inventées et les inventions médicales, détectées et comptées ; la lisibilité de la scène pour les sens (jauge par zone).

- **Mis à jour** : l'adoption au premier contact baisse un peu (22 → 18 sur 42), reste loin de la baseline (étape 16) ; le biais « tendance » du 8b ne se voit pas sur 3 pharmaciens (étape 16, 5 graines) ; le chargement des modèles prend un tiers du temps d'un run C1 (498 s sur 1 525 s, 56 bascules).
- **Nouveaux** : 69 % des prompts de C1 sont réduits pour tenir dans le contexte (602 sur 870, aucun dans le témoin) : les messages reçus saturent le prompt, il faut vérifier ce que la réduction retire (étape 16) ; le « tendance » des agents à règles vient de leur règle (imitation = tendance), ce qui pèse sur toute comparaison de sens entre C0 et C1 (étape 16) ; la convergence seule ne prouve pas B3 (question ouverte au binôme).
- **Durée** : un run C1 de 60 agents sur 60 jours prend 25 min (870 appels), mieux que prévu ; soit environ 1 h 15 projetée pour 120 agents sur 90 jours.

### Les tests

- **Calcul** : bornes de la convergence et de l'information mutuelle ; la permutation sépare des sous-cultures du hasard ; l'état de la population compte le dernier avis et cite ses décisions.
- **Service** : indicateurs publiés à chaque fin de jour, reproductibles ; les runs antérieurs reçoivent leurs fins de jour au rejeu.
- **Dérives** : chaque signe détecté ; correspondants tirés des seuls événements ; dérives publiées avec le jour.
- **Témoin** : aucun message remis, part de porteurs masquée ; chaque rôle dit comment il accueille un produit nouveau ; la trace remonte d'une valeur aux messages qui l'ont précédée.
- **Scène** : jauges et courbe appliquées depuis les messages du pont (`test_monde.gd`).

Résultat : 213 tests réussis (200 sans services, 7 avec Kafka et Redis, 1 Godot, 5 avec Ollama), aucune remarque de ruff.

### À retenir pour l'oral

- Le contexte commun se mesure avec deux nombres : à quel point un groupe s'accorde (convergence), et si les groupes pensent encore différemment (information mutuelle, avec sa p-valeur).
- Le témoin sans diffusion est la preuve : les échanges doublent l'adoption, mais le sens des agents LLM est déjà là sans eux. C'est la réponse honnête à Q2.
- Toute valeur affichée remonte aux décisions, puis aux messages : la chute du jour 7 vient d'une seule publication d'un créateur de contenu.
- Les petits modèles dérivent plus (alphabet, personnes inventées), et la dérive est comptée, pas cachée.

### Prochaine étape

Étape 15 — Mode asynchrone et cohérence temporelle : chaque agent agit à son rythme, latences réelles des modèles comprises (C4) ; place des chargements de modèles dans le temps simulé ; violations temporelles détectées et comptées sans barrière (C6) ; même scénario dans les deux modes et hypothèse formulée avant de mesurer (C5).

# Étape 15 — Mode asynchrone et cohérence temporelle

24 octobre 2026 · Exigences concernées : C3 (deux horloges), C4 (deux modes sélectionnables au lancement), C5 (comparer les deux modes, hypothèse avant la mesure), C6 (aucune information du futur, violations comptées), O10 (effet du modèle de temps sur le phénomène).

### Objectif

Ajouter le second mode d'exécution demandé par C4 : sans barrière, chaque agent agit à son rythme, latences réelles des modèles comprises. Il faut garder la cohérence temporelle (C6) alors que les agents ne décident plus au même instant, puis comparer les deux modes sur le même scénario (C5).

### Décisions du binôme

| Question | Décision |
|---|---|
| Le 8b, qui ne tient pas en VRAM avec les deux autres | **Neutraliser** : la journée garde deux phases (1.7b et 4b ensemble, puis 8b, ordre alterné chaque jour). L'heure de chaque phase repart de 08:30 et le chargement d'un modèle n'y compte pas. Un agent ne voit que ce qui est daté avant son passage ; ce qu'une phase suivante calcule trop tard pour lui est une **information manquée**, comptée, pas une violation. |
| Rythme | **Une décision par agent et par jour**, délai de réflexion inchangé : seul le modèle de temps change entre les deux modes, et le budget d'appels reste le même. |
| Agents à règles (sans latence) | **Heure tirée** avec la graine entre 08:30 et 12:00. |
| Hypothèse (C5), formulée avant de mesurer | **Les rapides donnent le ton** : petits modèles et agents à règles décident plus tôt, leurs messages touchent les autres le jour même ; l'adoption monte plus vite au début ; le sens penche vers celui des petits modèles ; le résultat principal tient dans les deux modes. |

### Comment marche une journée asynchrone

- **L'heure suit le calcul.** Pendant une phase, l'heure simulée avance avec le temps réel de calcul : 15 minutes simulées par seconde (`simulation.asynchrone.minutes_par_seconde`). Une seconde de chargement de modèle en est retirée (`HorlogePhase`), et l'heure ne recule jamais.
- **Chacun passe quand une place se libère.** Pour chaque palier, autant d'« ouvriers » que de requêtes simultanées permises font passer les agents un par un, dans un ordre tiré avec la graine (l'ordre alphabétique favoriserait toujours les mêmes). Au passage, l'agent perçoit le monde ; s'il n'est pas déclenché, il garde sa décision.
- **Deux heures par décision (C3).** `sim_time_occurred` est l'heure où l'agent commence à réfléchir, `sim_time_processed` celle où son modèle répond. Jusqu'ici, les deux étaient toujours égales.
- **La décision s'applique aussitôt.** Achat (premier arrivé, premier servi), changement de porteur, messages déposés : un message de zone (sans délai) ou direct (2 h) peut être lu le jour même par quelqu'un qui passe après.
- **Pas d'information du futur (C6).** La perception à l'heure s ne compte que les décisions et les messages datés de s ou avant, même si une phase les a calculés plus tard. Le contrôle des causes citées reste actif.
- **Publication en fin de journée**, triée par heure simulée : la console, l'historique et Godot voient le jour dans l'ordre de ses heures.

> **Une information manquée**, c'est un message livré à un agent *avant* son passage, mais calculé *après* lui, parce qu'il vient de la phase suivante. L'agent le lira à sa prochaine décision. C'est le prix de la contrainte de VRAM, et il est mesuré : `horloge.fin_jour` (version 1.1) en donne le nombre et cite les messages concernés ; le bilan le totalise et le rejeu le recalcule. En cours d'étape, un test a montré qu'une égalité à la minute près (deux agents à 08:30) comptait à tort comme un retard : la condition est désormais stricte.

### Ce qui a été fait

| Fichier | Rôle |
|---|---|
| `sim/orchestrateur.py` | La journée se partage entre un début et une fin communs, et deux façons de décider : synchrone (inchangée) ou asynchrone (phases, ouvriers par palier, perception datée, application immédiate, informations manquées, publication triée). |
| `sim/horloge.py` | `HorlogePhase` : heure d'une phase, liée au temps réel, hors chargements, jamais en recul, bornée avant le soir. |
| `sim/config.py`, `configs/default.yaml` | Section `simulation.asynchrone` (début, minutes par seconde, fenêtre des agents à règles), avec des valeurs par défaut pour relire les anciens manifestes. |
| `configs/dev-asynchrone.yaml`, `baseline-dev-asynchrone.yaml` | Les mêmes mondes que `dev` et `baseline-dev`, en mode asynchrone. |
| `bus/events.py` | `horloge.fin_jour` 1.1 : informations manquées, et les messages concernés dans `caused_by` (F6). |
| `replay/bilan.py`, `replay/rejeu.py` | Le rejeu recalcule les informations manquées ; un compteur absent d'un ancien bilan vaut sa valeur initiale. |
| `replay/horloges.py` | Les deux horloges d'un run : heure de décision et durée de réflexion par palier, messages lus le jour même et par qui, délai entre émission et lecture, informations manquées par rôle. |

### Résultat 1 — la baseline C0 (5 graines, 60 agents, 60 jours)

Sans modèle de langage, l'effet du seul modèle de temps se mesure proprement, et la baseline asynchrone reste reproductible (deux runs de même graine identiques). Détail : `docs/mesures/comparaison_2026-10-24_1800.md`.

| Porteurs (moyenne ± écart-type) | Jour 1 | Jour 5 | Jour 12 | Jour 20 | Jour 30 |
|---|---|---|---|---|---|
| C0 synchrone | 3,6 ± 0,9 | 6,8 ± 2,9 | 24,2 ± 15,9 | 50,2 ± 7,2 | 50,8 ± 2,8 |
| C0 asynchrone | 4,8 ± 2,2 | **12,6 ± 14,8** | **36,6 ± 20,2** | 55,4 ± 3,0 | 57,0 ± 2,3 |

Sans barrière, ceux qui passent tard dans la matinée imitent ceux qui ont adopté plus tôt le jour même : la courbe se décale de 3 à 4 jours vers le début, le pic arrive plus tôt (jours 15 à 25 contre 32 à 45) et l'écart entre graines grandit au démarrage. L'état final est le même : la saturation, déjà connue.

### Résultat 2 — C1, les agents LLM (graine 42, même scénario)

Détail : `docs/mesures/indicateurs_2026-10-24_1836.md`, `comparaison_2026-10-24_1836.md` et `horloges_2026-10-24_1838.md`.

| Mesure | Synchrone | Asynchrone |
|---|---|---|
| Porteurs aux jours 5, 12, 20, 30, 60 | 22, 40, 42, 45, 49 | 23, 42, 34, 41, 46 |
| Convergence C, jour 1 → jour 60 | 0,46 → 0,25 | 0,45 → 0,14 |
| Information mutuelle, fin (p) | 0,48 (0,005) | 0,46 (0,005) |
| Messages, dont lus le jour même | 110, 0 | 131, **29** |
| Délai médian émission → lecture | 48 h | 45 h |
| Informations manquées | – | **42** (22 messages, dont 14 de pharmaciens) |
| Appels, durée réelle, chargements | 870, 1 525 s, 498 s | 867, 1 545 s, 476 s |
| Durée cumulée des appels | 2 150 s | **3 662 s** |
| Dérives des décisions LLM | 36 (4,1 %) | 55 (6,3 %), dont 32 personnes inventées |

| Palier (asynchrone) | Heure médiane de décision (p10 – p90) | Réflexion médiane | Messages émis |
|---|---|---|---|
| Règles | 10:12 (08:48 – 11:36) | 0 min | 2 |
| 1.7b | **10:42** (09:10 – 13:30) | 37 min | 4 |
| 4b | 10:27 (09:24 – 12:31) | 99 min | 68 |
| 8b | 10:15 (09:15 – 12:28) | 105 min | 57 |

### L'hypothèse confrontée aux mesures

- **« Les rapides donnent le ton » : non, pour deux raisons que l'hypothèse ignorait.** D'abord, le rythme réel d'un agent, c'est sa latence *plus la file d'attente* de son modèle : le 1.7b répond vite, mais une trentaine d'agents se partagent 2 places, si bien qu'il décide le plus tard (10:42 en médiane). Ensuite, les agents rapides ne parlent presque pas (4 messages en 60 jours) : les voix du monde sont les créateurs et journalistes (4b) et les pharmaciens (8b). Les 29 lectures du jour même vont surtout des créateurs vers les baigneurs (26).
- **« L'adoption monte plus vite au début » : oui pour la baseline, pas visible chez les agents LLM.** C0 gagne 3 à 4 jours sur 5 graines. En C1, l'avance se limite à 1 ou 2 porteurs aux jours 5 et 12, puis l'asynchrone passe en dessous ; sur une seule graine, ces écarts restent dans le bruit d'un run LLM. La raison tient au délai de réflexion : un message attend le plus souvent 2 à 3 jours avant de compter (délai médian 45 h contre 48 h), quelques heures de plus ou de moins dans la journée n'y changent presque rien.
- **« Le résultat principal tient dans les deux modes » : oui pour le sens.** Rôle par rôle, le sens final est le même : baigneurs « efficace » 12 sur 17 dans les deux modes, sceptiques « banal » 5 sur 8, habitués « efficace » 6 sur 8 contre 7 sur 8. L'information mutuelle finit à 0,46 contre 0,48. Pour l'adoption, il faudrait un témoin asynchrone pour le dire (étape 16).
- **Le coût caché de l'asynchrone** : même nombre d'appels et même durée réelle, mais chaque appel dure 70 % de plus, parce que le 1.7b et le 4b se partagent la carte graphique au même moment. La « latence réelle » mesurée en asynchrone inclut donc cette concurrence.
- **Le prix des phases** : 42 informations manquées contre 29 lectures du jour même. La séparation du 8b retarde plus de messages que l'asynchronisme n'en accélère ; 14 des 22 messages concernés viennent des pharmaciens.

### Vérifications

- 0 incident, 0 relance de modèle, 0 repli, 0 violation temporelle ; 2 961 événements publiés et enregistrés ; rejeu fidèle (bilan identique).
- Le 42 a été compté avec la règle du run (livré au plus tard au passage) ; la règle stricte, corrigée après ce run, peut donner un peu moins si des cas étaient simultanés à la minute près.
- 6 relances d'embedding en asynchrone, contre 0 en synchrone : 1.7b, 4b et bge-m3 travaillent en même temps, Ollama est plus chargé (constat de l'étape 17).
- Une tranche LLM asynchrone (4 agents, 5 jours) : 12,6 s de chargement du 8b exclues de l'heure, rejeu fidèle.

### Constats

- **Nouveaux** : le rythme réel est fixé par la file d'attente autant que par la taille du modèle (le 1.7b décide le plus tard) ; les appels durent 70 % de plus en asynchrone (concurrence sur la carte graphique) ; plus de personnes inventées en asynchrone (32 contre 13, une seule graine) ; 42 informations manquées, surtout des messages de pharmaciens. Tous à mesurer sur 5 graines à l'étape 16.
- **Mis à jour** : Ollama sous charge (6 relances d'embedding en asynchrone) ; deux tests du RAG échouent parfois dans la suite complète avec les tests Ollama, jamais seuls ni sans eux (étape 17).

### Les tests

- **Horloge de phase** : suit le temps réel, ignore les chargements, ne recule jamais, bornée avant le soir.
- **Lecture le jour même** d'un message de zone par un agent de l'autre phase ; un message daté de la fin d'un long appel attend le lendemain.
- **Deux heures par décision** et publication du jour dans l'ordre de ses heures.

- **Perception datée** : un agent ne voit un porteur que si sa décision est datée d'avant son passage, même calculée plus tôt (C6) ; une cause jamais reçue est toujours détectée.
- **Informations manquées** comptées, citées par la fin du jour et recalculées par le rejeu ; une égalité à la minute n'en est pas une.
- **Agents à règles** à leur heure tirée, reproductible ; **chargement** exclu de l'heure ; **parallélisme** par palier respecté, une décision par agent et par jour.

Résultat : 225 tests (dont 12 nouveaux), **tous réussis** sans les tests Ollama (3 lancements sur 3) ; dans la suite complète, un test du RAG a échoué une fois puis réussi seul (voir les constats). Aucune remarque de ruff.

### À retenir pour l'oral

- Deux modes au choix dans la configuration. En asynchrone, l'heure simulée suit le temps de calcul, et chaque décision porte l'heure où l'agent commence à réfléchir et celle où il finit.
- La carte graphique impose deux phases ; leur coût est mesuré (informations manquées), pas caché.
- Q6 du jury : le résultat principal tient dans les deux modes pour le sens. La baseline montre que le modèle de temps accélère l'imitation ; chez les agents LLM, c'est le délai de réflexion qui domine.
- L'hypothèse était fausse en partie, et c'est instructif : « rapide » ne veut pas dire « tôt » (la file d'attente compte), et ce ne sont pas les rapides qui parlent.

### Prochaine étape

Étape 16 — Campagnes d'expériences : configurations C0 à C3, au moins 5 graines chacune (E1, E2), runs de 90 jours, synchrone contre asynchrone avec témoins (C5), coûts (E4, E5) ; et les constats qui en dépendent : adoption au premier contact, saturation, durée des flacons, prompts réduits, bascules de modèles.